

make_cdB Noise Characterizer User Guide

Product Version 26.10
May 2026

©2011-2026 Cadence Design Systems, Inc. All rights reserved.

Printed in the United States of America.

Cadence Design Systems, Inc. (Cadence), 2655 Seely Ave., San Jose, CA 95134, USA.

Trademarks: Trademarks and service marks of Cadence Design Systems, Inc. (Cadence) contained in this document are attributed to Cadence with the appropriate symbol. For queries regarding Cadence's trademarks, contact the corporate legal department at the address shown above or call 1-800-862-4522.

All other trademarks are the property of their respective holders.

Restricted Print Permission: This publication is protected by copyright law and international treaties and contains trade secrets and proprietary information owned by Cadence. Unauthorized reproduction or distribution of this publication, or any portion of it, may result in civil and criminal penalties. Except as expressly permitted below, this publication may not be copied, reproduced, modified, published, uploaded, posted, transmitted, or distributed in any way, including automated processes, training or services involving a large language model, foundation model, deep machine learning, generative artificial intelligence, or any other similar process or technology, without prior written permission from Cadence. Unless otherwise agreed to by Cadence in writing, customers are granted permission to print one (1) hard copy of this publication subject to the following conditions:

1. The publication may be used solely for personal, informational, and noncommercial purposes;
2. The publication may not be modified in any way;
3. Any copy of the publication or portion thereof must include all original copyright, trademark, and other proprietary notices and this permission statement; and
4. Cadence reserves the right to revoke this authorization at any time, and any such use shall be discontinued immediately upon written notice from Cadence.

Disclaimer: Information in this publication is subject to change without notice and does not represent a commitment on the part of Cadence. Except as may be explicitly set forth in such agreement, Cadence does not make, and expressly disclaims, any representations or warranties as to the completeness, accuracy or usefulness of the information contained in this document. Cadence does not warrant that use of such information will not infringe any third party rights, nor does Cadence assume any liability for damages or costs of any kind that may result from use of such information. Cadence is committed to using respectful language in our code and communications. We are also active in the removal and/or replacement of inappropriate language from existing content. This product documentation may however contain material that is no longer considered appropriate but still reflects long-standing industry terminology. Such content will be addressed at a time when the related software can be updated without end-user impact.

Restricted Rights: Use, duplication, or disclosure by the Government is subject to restrictions as set forth in FAR52.227-14 and DFAR252.227-7013 et seq. or its successor.

Contents

<u>Preface</u>	6
<u>About This Manual</u>	7
<u>Documentation Conventions</u>	8
<u>Text Command Syntax</u>	8
 1	
<u>Noise Library Characterization</u>	9
<u>Overview</u>	12
<u>Performing Noise Characterization</u>	16
<u>User-Defined Noise Model - An Overview</u>	24
<u>Additional Information</u>	29
 2	
<u>make_cdB Commands</u>	42
<u>bb_set_bidi</u>	44
<u>bb_set_input</u>	46
<u>bb_set_output</u>	48
<u>create_ecsm_si_lib</u>	49
<u>generate_cell_lib</u>	50
<u>merge_cdb</u>	57
<u>message_handler</u>	58
<u>pcdb</u>	70
<u>print_net</u>	72
<u>read_dotlib</u>	73
<u>redirect_output</u>	74
<u>regenerate_cell_lib</u>	75
<u>report_library</u>	78
<u>reset_redirection</u>	81
<u>set_cell_input_slewlis</u> t	82
<u>set_message_hook</u>	83

make_cdB Noise Characterizer User Guide

<u>set_net</u>	86
<u>set_port</u>	88
<u>set_run_mode</u>	90
<u>set_simwrap_mode</u>	91
<u>set_supply</u>	92
<u>set_supply_char_vals</u>	93
<u>validate_cell_lib</u>	95

Global Variables

<u>bind_supply_to_port</u>	98
<u>cabstol</u>	98
<u>calculate_switching_history</u>	98
<u>caplimit</u>	99
<u>case_sensitive</u>	99
<u>ccc_max_fets</u>	99
<u>ccc_max_fets_for_vivochar</u>	100
<u>ccc_max_nets</u>	100
<u>ccc_max_ports</u>	100
<u>do_prop_trans</u>	101
<u>large_ccc_max_fets</u>	101
<u>max_patterns</u>	101
<u>max_ports</u>	101
<u>parse_vsrc_as_resvalue</u>	103
<u>port_gnd_r</u>	103
<u>port_vdd_r</u>	103
<u>port_vh_noise_tolerance</u>	103
<u>port_vl_noise_tolerance</u>	104
<u>print_rt_memu</u>	104
<u>prop_input_slew_cell</u>	104
<u>rabstol</u>	104
<u>scale_cdb</u>	105
<u>simwrap_spectre_cmd_line_options</u>	105
<u>slew_fall_high</u>	105
<u>slew_fall_low</u>	105
<u>slew_rise_high</u>	106
<u>slew_rise_low</u>	106
<u>spice_hdif</u>	106

make_cdB Noise Characterizer User Guide

<u>spice_scale</u>	107
<u>spice_scalm</u>	107
<u>tmpdir</u>	107

Preface

- [About This Manual](#) on page 7
- [Documentation Conventions](#) on page 8

About This Manual

This manual describes how to use the `make_cdB` noise characterizer utility to generate a cdB noise library from transistor-level netlists and SPICE models, including memory characterization. It also describes the `make_cdB` commands and global variables.

Documentation Conventions

Text Command Syntax

The list below defines the syntax conventions used for the CeltIC NDC text interface commands.

<code>literal</code>	Nonitalic words indicate keywords you enter literally. These keywords represent command or option names.
<i>arguments and options</i>	Words in italics indicate user-defined arguments or information for which you must substitute a name or a value.
	Vertical bars (OR-bars) separate possible choices for a single argument.
[]	Brackets indicate optional arguments. When used with OR-bars, they enclose a list of choices from which you can choose one.
{ }	Braces indicate that a choice is required from the list of arguments separated by OR-bars. Choose one from the list.
	<code>{ argument1 argument2 argument3 }</code>
{ }	Braces, used in Tcl commands, indicate that the braces must be typed in.
...	Three dots (...) indicate that you can repeat the previous argument. If the three dots are used with brackets (that is, [argument]...), you can specify zero or more arguments. If the three dots are used without brackets (argument...), you must specify at least one argument.
#	The pound sign precedes comments in command files.

Noise Library Characterization

■ Overview

- ❑ make_cdB Tcl Scripts
- ❑ cdB Contents
 - Propagated Slew
 - Arc Current Models
- ❑ cdB Noise Library Format
 - cdB Noise Library Structure
 - Characterized Noise Data
 - Cell Information
 - Cell Input Data
 - Cell Output Data
 - Cell Bi-Directional Data

■ Performing Noise Characterization

- ❑ Running make_cdb
- ❑ Specifying Power Supply Names
- ❑ Setting the Default Power Supplies
 - Setting Global and Local Power Supplies
 - Setting Multiple Power Supplies
- ❑ Identifying the I/Os
 - Extracting Port Directions from Timing Library Files
 - Extracting Port Directions Using set_port

make_cdB Noise Characterizer User Guide

Noise Library Characterization

- ❑ [Generating the cdB Noise Library](#)
- ❑ [Regenerating the cdB Noise Library](#)
 - [Guidelines for cdB Regeneration](#)
- ❑ [Using Model Libraries in make_cdB](#)
 - [Sample SPICE Netlist File](#)
- ❑ [Merging Multiple Noise Libraries](#)
- ❑ [Printing Cell and UDN Information](#)
- [User-Defined Noise Model - An Overview](#)
 - ❑ [Glitch Injection from UDN](#)
 - ❑ [UDN Specification Allows Quiet Net Settings for Floating Nets](#)
 - ❑ [Example UDNs](#)
 - [Example of Single UDN Driver Set Quiet](#)
 - [Example of Single UDN Receiver Set Quiet](#)
 - [Example of Multi-Driver with UDN Pin Set Quiet](#)
 - [Example of Multi-Receiver with UDN Pin Set Quiet](#)
- [Additional Information](#)
 - ❑ [Slew Propagation](#)
 - [Characterization](#)
 - [Analysis](#)
 - ❑ [Constructs in Arc Current Models](#)
 - ❑ [Characterizing Different Cell Types](#)
 - [Tiehi and Tielo Cells](#)
 - [RAM Cells](#)
 - [User-Defined Noise Models](#)
 - [Filler/Antenna/Decoupling Cells](#)
 - [Sequential and Combinational Cells](#)
 - [Level-Shifter Cells](#)

make_cdB Noise Characterizer User Guide

Noise Library Characterization

- ❑ Multi-Voltage Characterization
- ❑ Generating an ECSM-Noise Liberty Library
- ❑ Guidelines for Characterizing Large Cells

Overview

This chapter describes how to use the `make_cdB` noise characterizer utility to generate a cdB noise library from transistor-level netlists and SPICE models. In addition, it discusses memory characterization.

Before the signal integrity analyzer performs a comprehensive crosstalk analysis of a cell-based digital circuit, each cell in a cell library can be characterized by the `make_cdB` utility. Alternatively, cells can be modeled using black-boxes, which include user-defined noise (UDN) models or timing libraries (.lib). However, in this case, crosstalk analysis is not as accurate as compared to using data characterized by `make_cdB`.

The `make_cdb` utility analyzes the transistor-level netlist of a cell for noise effects as seen from its input and output ports. It computes the essential information needed for noise analysis and encapsulates it in a cell-level model. The cell-level model contains the subcircuit (subckt) related information for the cell along with the noise and delay characteristics of the ports of the cell.

The characterized noise library typically has a `.cdb` extension. This resultant cell library (`.cdb` file) contains a cell-level view and a transistor-level view. The cell-level view contains data, such as pin capacitance, calibrated input noise threshold, and nonlinear output drive strength. The transistor-level view contains the transistor cell subcircuit. However, if the cell is large, the transistor-level view contains only the peripheral transistors connected to ports of the cell. For more information on characterizing large cells, see [RAM Cells](#) on page 33.

make_cdB Tcl Scripts

The `make_cdB` utility supports a Tcl interface. Most `make_cdB` scripts contain commands in the following sequence:

```
set_supply
set_net
read_dotlib
generate_cell_lib
```

The following example illustrates the contents of a sample Tcl script, which shows the use of the `set_supply` and `generate_cell_lib` commands.

```
set_parm port_vdd_r 1
set_parm port_gnd_r 1
#Set the global supply. VDD is set to 1.5V and GND is set to 0.0V.
set_supply -vdd 1.5 -gnd 0
#Set non-global supplies.
set_net -vdd 3 vdd3
```

make_cdB Noise Characterizer User Guide

Noise Library Characterization

```
# Pick up the pin directions from the wc.lib file.
read_dotlib wc.lib
# Generate the CDB for the cell, XOR.
# Define the transistor level netlist and models for XOR in the SPICE main.spi file.
# Output the CDB in the cells.cdB file.
generate_cell_lib \

-vdd { vdd VDD vdd3 VDD3 } \

-gnd { gnd GND 0 } \

-cell_list { XOR } \

-file_list { main.spi } \

-file cells.cdB
```



Tip

Specify the correct supply voltage and other parameters that will be used in any subsequent noise analysis run. If there are inconsistencies between the parameters specified for characterization and those specified for subsequent analysis runs, the noise analysis results will be incorrect.

cdB Contents

This section describes the cdB contents, which include propagated slew and arc current models.

Propagated Slew

A cdB contains propagated slew. Slew propagation calculations are used in situations where multistage cells drive the nets. These calculations enable you to propagate the slews available at the inputs of the cell down to the driving channel-connected component (CCC) inputs. For more information about slew propagation, see [Slew Propagation](#) on page 30.

Arc Current Models

Arc current models, which are used for low effort crosstalk delay modeling during signal integrity analysis, represent the current drawn by the driver as a function of input and output voltage. There are unique tables for each transition and from each last logic stage input (input pin or internal node) to the output. At the input pin, arc current models are used for path based alignment during signal integrity analysis. At the output pin, arc current models are used for

low-effort modeling of aggressor and victim drivers. For more information about arc current models, see [Constructs in Arc Current Models](#) on page 31.

cdB Noise Library Format

To perform noise-on-delay and glitch analysis, the signal integrity analyzer requires a noise view for each cell in the circuit. The creation of these noise views is done using the `make_cdB` utility. This utility automatically characterizes a group of cells or macro blocks to create a binary cdB noise library. The ASCII equivalent format for these noise views is described in this section.

→ To create a cdB noise library in ASCII format, use the `-text` option of the `generate_cell_lib` command.

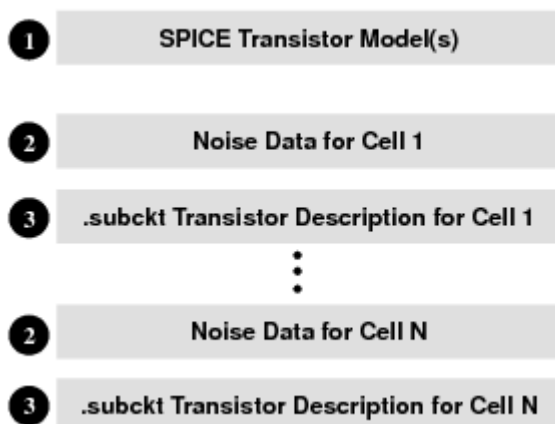
cdB Noise Library Structure

The cdB noise library structure includes:

- SPICE models used for characterizing the cdB noise library in BSIM3, BSIM4 or MOS9 formats.
- Characterized noise data for each cell.
- Transistor-level (`.subckt`) description of each cell in SPICE format.

The figure below shows the cdB noise library structure.

Figure 1-1 cdB Noise Library Structure



make_cdB Noise Characterizer User Guide

Noise Library Characterization

The data in sections 1 and 3 are copies of the input data provided to the `make_cdB` utility. The signal integrity analyzer uses this data to perform a complete transistor-level simulation for certain noise calculations. The characterized noise data for a single cell (section 2) is described in the following sections.

Characterized Noise Data

The characterized noise data includes information about the cell, input pins, output pins, and bidirectional pins.

This section discusses the following topics:

- [Cell Information](#)
- [Cell Input Data](#)
- [Cell Output Data](#)
- [Cell Bi-Directional Data](#)

Cell Information

`bb_is_cell`

Flag to indicate that this is a cell with a transistor-level view.

Cell Input Data

`bb_set_input [-vhtolerance float] [-vltolerance float] [-cap float] {ports}`

Marks the specified ports as UDN inputs.

For more details, see [bb_set_input](#).

Cell Output Data

`bb_set_output [-vddres float] [-gndres float] [-canfloat] [-cap float] [-vhnoise {time_value_pairs}] [-vlnoise {time_value_pairs}] {ports}`

Marks the specified ports as a UDN output.

For more details, see [bb_set_output](#).

make_cdB Noise Characterizer User Guide

Noise Library Characterization

Cell Bi-Directional Data

```
bb_set_bidi [-vhtolerance float] [-vltolerance float] [-cap float]
            [-cannotfloat] [-vddres float] [-gndres float]
            [-vhnoise {time_value_pairs}] [-vlnoise {time_value_pairs}] {ports}
```

Marks the specified ports as a bidirectional input/output port. This command has most of the features of the `set_input` and `set_output` commands. The defaults for inputs and outputs are used for bidirectionals.

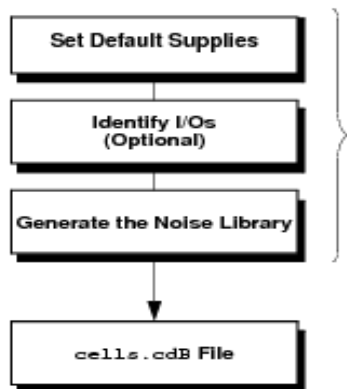
For more details, see `bb_set_bidi`.

Performing Noise Characterization

This section describes the tasks you perform to generate the cdB noise library.

The figure below shows the `make_cdB` flow. The resultant `.cdB` file is required as input for performing signal integrity analysis.

Figure 1-2 make_cdB Flow



EXAMPLE:

```
set_supply -vdd 1.5 -gnd 0
set_net -vdd 3 vdd3
read_dotlib wc.lib
generate_cell_lib \
    -vdd { vdd VDD vdd3 VDD3 } \
    -gnd { gnd GND 0 } \
    -cell_list { XOR } \
    -file_list { main.spi } \
    -file cells.cdB
```

where main.spi contains:

```
* title
.lib model_lib.spi tt
.include cells.spi
.temp 0
```

Running make_cdb

You can run the `make_cdB` utility in interactive or batch mode from the UNIX prompt.

To run the `make_cdB` utility in interactive mode, use the following command:

```
% make_cdb
```


make_cdB Noise Characterizer User Guide

Noise Library Characterization

To run the `make_cdB` utility in batch mode, use the following command:

```
% make_cdb cmdfile.tcl |& tee logfile.log
```

where, the `cmdfile.tcl` contains valid `make_cdB` or Tcl commands and the output of the `make_cdb` command is replicated to the `logfile.log` file using the `tee` command.

The complete syntax of the `make_cdb` command is as follows:

```
make_cdb
    [-v]
    [-c]
    [-check]
    [cmdfile]
```

Parameters

<code>-c</code>	Provides backward compatibility when using netlist view handles in commands.
<code>-check</code>	Checks the Tcl code for obvious errors before running <code>make_cdB</code> . Running this check can detect typos in the Tcl code or detect commands that are given in wrong sequence, which could otherwise be overlooked during a <code>make_cdB</code> run and waste valuable time. You can use this option from the command line as follows: <code>make_cdb -check batch_file.tcl</code>
<code>cmdfile</code>	Specifies the optional command file that contains <code>make_cdB</code> commands to run in batch mode.
<code>-v</code>	Prints the <code>make_cdB</code> version in use and the copyright information.

Specifying Power Supply Names

The power supply names must be specified using the `-vdd` and `-gnd` options. There are no default power supply names, so the default supply voltage is applied as specified in the `set_supply` command. Additional voltages can be specified on a local basis using the `set_net` command.

Setting the Default Power Supplies

There are two types of default power supplies: global and local. Use the `set_supply` command to set the global power supply and the `set_net` command to set the local power supply.

Setting Global and Local Power Supplies

Global power supplies are used during simulation to set circuit voltages.

Power supply nodes must be identified for the `generate_cell_lib` command to work properly. These nodes help to identify capacitors connected to supply nets.

→ To set the global power supplies, use the `set_supply` command.

Local power supplies are used when multiple voltages are desired.

→ To set the local power supplies, use the `set_net` command.

The following example illustrates how the `set_supply` and `set_net` commands are used to set the global and local power supplies, respectively:

```
set_parm port_vdd_r 1
set_parm port_gnd_r 1
set_supply -vdd 1.5 -gnd 0
set_net -vdd 3 vdd3
read_dotlib wc.lib
generate_cell_lib \

    -vdd { vdd VDD vdd3 VDD3 } \

    -gnd { gnd GND 0 } \

    -cell_list { XOR } \

    -file_list { main.spi } \

    -file cells.cdB
```

In the above example:

The `set_supply` command specifies the global power supply. The `-vdd` option specifies the nominal supply voltage as 1.5V and the `-gnd` option specifies the value of nominal ground voltage as 0.0V.

The `set_net` command specifies the local power supply. The `-vdd` option specifies the value, 3.0V, for the net, `vdd3`.

Setting Multiple Power Supplies

You set multiple power supplies if a cell can have multiple operating voltages, such as the one used in multiple supply multiple voltage (MSMV) designs. This is not required for proper IR-drop analysis. To set multiple power supplies in `make_cdB`, use multiple `set_supply` commands.

Note: For more information about setting multiple power supplies, see [Multi-Voltage Characterization](#) on page 39.

Identifying the I/Os

During noise characterization, the `make_cdB` utility automatically extracts I/O port directions in the following sequence:

1. As specified in the timing library file.
2. As specified by the `set_port` command.

Extracting Port Directions from Timing Library Files

To extract the port directions from a ASCII cell timing library, use the `read_dotlib` command before the `generate_cell_lib` command, as shown in the following example:

```
set_parm port_vdd_r 1
set_parm port_gnd_r 1
```

```
set_supply -vdd 1.5 -gnd 0
set_net -vdd 3 vdd3
```

#The `read_dotlib` command extracts pin directions from the `wc.lib` file.

```
read_dotlib wc.lib
generate_cell_lib \

-vdd { vdd VDD vdd3 VDD3 } \

-gnd { gnd GND 0 } \

-cell_list { XOR } \
```

make_cdB Noise Characterizer User Guide

Noise Library Characterization

```
-file_list { main.spi } \  
  
-file cells.cdB
```

Extracting Port Directions Using set_port

Ports connected to transistor gates are marked as inputs and ports connected to transistor channels are marked as outputs.

Note: All input ports that are connected to transistor channels and all bidirectional ports must be declared to the `make_cdB` tool by using the `set_port` command.

To set the port directions manually, use the `set_port` command, as in the following example:

```
set_parm port_vdd_r 1  
set_parm port_gnd_r 1  
  
set_supply -vdd 3.3 -gnd 0  
set_port -dir in NOR2 A1  
set_port -dir in NOR2 B1  
set_port -dir out NOR2 O  
generate_cell_lib \  
  
    -cell_list { NOR2 } \  
  
    -vdd {vdda} -gnd {vssa} \  
  
    -file_list { cells.sp ../models/model_pt.sp} \  
  
    -file cells.cdB
```

In the above example:

The `set_port` command is used with the `-dir` option to specify the input and output pins of the `NOR2` cell.

A1 and B1 are input pins of the `NOR2` cell.

O is the output pin of the `NOR2` cell.

make_cdB Noise Characterizer User Guide

Noise Library Characterization

Note: Because SPICE does not specify port directions, the `make_cdB` utility assumes all channel-connected ports to be cell outputs. If a cell contains channel (unbuffered) inputs, you must use the `set_port` command to override the default port direction.

Generating the cdB Noise Library

The `generate_cell_lib` command generates the cdB noise library that can be used to represent the cells while performing signal integrity analysis. For more information about this command, see `generate_cell_lib` in the *make_cdB Commands* chapter.

If the error log reports missing models when you generate the library, in most cases, these errors are reported due to a mismatch in model names. For example, your netlist may contain devices called `mn1`, `mn2` . . . and `mp1`, `mp2`, and so on. While in your models file, these devices may be called `nmos` and `pmos` respectively. To correct these errors, you can use `.equiv pmos=mp1 nmos=mn1` in your SPICE netlist to make the names match (preferred). You can also do a global substitution in the model file to match the name in the netlist.

Regenerating the cdB Noise Library

To convert an old cdB noise library file to the latest cdB noise library file version, use the `regenerate_cell_lib` command, as shown in the following example:

```
set_parm port_vdd_r 1
set_parm port_gnd_r 1

set_supply -vdd 1.5 -gnd 0

set_net -vdd 1.8 VDD
set_net -vdd 3.0 VDD3
set_net -gnd 0 GND

regenerate_cell_lib \

-infile old.cdB \
```

make_cdB Noise Characterizer User Guide

Noise Library Characterization

```
-outfile new.cdB \
```

```
-version_string rechar-to-4.3.2
```

The `regenerate_cell_lib` command supports multi-voltage cdB regeneration. This means that you can regenerate an existing cdB to create a new cdB with cells characterized at different voltage values specified during regeneration. For example:

```
set_run_mode 12
```

```
set_supply -vdd 3.3 -gnd 0.0
```

```
set_supply -vdd 1.7 -gnd 0.0
```

```
set_net -vdd 2.0 { VDD2 }
```

```
set_net -nomvdd 3.3 -vdd 3.3 VDD
```

```
set_net -nomvdd 1.7 -vdd 1.7 VDD
```

```
set_port -dir in TOP {IN1}
```

```
set_port -dir out TOP {OUT1}
```

```
set_port -dir out TOP {OUT2}
```

```
set_port -dir out TOP {OUT3}
```

```
regenerate_cell_lib -infile test.cdB.txt -text
```

In the above example:

- The input cdB noise library `test.cdB` will be regenerated using new voltage values.
- Recharacterization will be performed at 3.3 and 1.7 supply voltage values.
- When performing recharacterization using the supply voltage of 3.3V, net `VDD` will be at 3.3V but net `VDD2` will be at 2.0V.
- When performing recharacterization using the supply voltage of 1.7V, net `VDD` will be at 1.7V but net `VDD2` will be at 2.0V.

Guidelines for cdB Regeneration

The following guidelines will help you during cdB regeneration:

- UDNs are preserved in their original form. No recharacterization is done.
- If you need to reset any of the parameter settings for recharacterization, use the `set_parm` parameters before running this command.
- You should always set the supply nets properly using the `set_supply` command. If the old cdB noise library file is older than `make_cdB` version 4.1, you must use the `set_supply` command to extract the supply voltage information. However, the `make_cdB` utility automatically extracts the supply voltage information from old cdB noise library files generated with `make_cdB` version 4.1 or newer.
- If the old cdB noise library file is encoded, you cannot generate a text output file for the new cdB noise library file.

Using Model Libraries in make_cdB

To use model libraries, create a main SPICE netlist that contains `.include` and `.lib` statements and load this file into `make_cdB` using the `-file_list` option of the `generate_cell_lib` command.

Sample SPICE Netlist File

```
* title .lib modelFile typ .include netlistFile
.temp myTemp
.lib myLib modelName
.inc mySpice
```

Merging Multiple Noise Libraries

To merge multiple noise libraries into a single noise library, use the `merge_cdb` command.

Note: If duplicate definitions are found, the first definition is kept and all others are ignored.

Printing Cell and UDN Information

To print information about cells and user-defined noise (UDN) models that are inside cdB noise libraries, use the `pcdb` command.

Note: The cell or UDN name and port list are printed with port direction.

User-Defined Noise Model - An Overview

A User-Defined Noise (UDN) model is created to allow noise analysis of a design that is not yet complete or contains non-digital circuitry. During analysis, a UDN input port is treated as a noise output port, and a UDN output port is treated as a noise input port to the surrounding circuit.

The UDN's are loaded along with the cdB's (`read_lib -cdb {}`).

You can mark certain ports as UDN input/output, or as bi-directional input/output ports. The following UDN commands can be used:

- `bb_set_bidi`
- `bb_set_input`
- `bb_set_output`

Glitch Injection from UDN

Noise can be injected in the circuit through UDNs by placing a glitch waveform on the output pins of a UDN.

The following UDN syntax includes noise injection:

```
.bbox mydrv I ZN
    bb_is_udn
    bb_set_input -vhtolerance 0.415 -vltolerance 0.415 -cap 3.44e-15 {I}
    bb_set_output -vddres 196.804679228 -gndres 220.565530019 -cap
    2.567848e-15 -vlnoise {0 0 333e-12 0.300 1000e-12 0} -vhnoise {0 1.05
    333e-12 0.750 1000e-12 1.05} {ZN}
    .endbb
```

This will inject both a VL and VH noise waveform into the circuit, wherever the ZN pin connects.

UDN Specification Allows Quiet Net Settings for Floating Nets

The quiet net specification inside a UDN indicates the following:

When the input pin of the UDN is set as quiet, the receiver input peak (RIP) is not checked.

make_cdB Noise Characterizer User Guide

Noise Library Characterization

When the output pin of the UDN is set as quiet and the pin is the only driver for a victim net, the software uses a large Rhold and the driver's pin cap, and reports the glitch.

When the output pin of the UDN is set as quiet and there are other drivers available for the victim net the other valid drivers will be used for glitch analysis and reporting.

When all the drivers of a net are set as quiet with UDN specifications then the driver will be modeled with a large Rhold and the driver's pin cap will be used for glitch analysis and reporting.

When the output pin of the UDN is set as quiet and it drives an attacker it is held constant using a large Rhold.

Example UDNs

To establish the UDN definition, create the following `inv.udn` file:

```
.bbox INVUDN BI BO
  bb_set_input -vhtolerance 0.2 -vltolerance 0.2 -cap 100e-15 BI;
  bb_set_output -vddres 1e3 -gndres 1e3 BO;
.endbb
```

The following example shows sample UDN syntax:

```
#quiet attacker
.bbox INVP3
+ Y VDD VSS A
bb_set_input -vhtolerance 0.26 -vltolerance 0.26 -cap 2.29e-15 A
bb_set_output -vddres 8000 -gndres 9000 -cap 4.01e-16 -quiet attacker Y
.endbb
#quiet victim
.bbox INVP16
+ Y VDD VSS A
bb_set_input -vhtolerance 0.26 -vltolerance 0.26 -cap 2.29e-15 -quiet
victim A
bb_set_output -vddres 8000 -gndres 9000 -cap 4.01e-16 Y
.endbb
#quiet both attacker and victim
.bbox INVP16
+ Y VDD VSS A
bb_set_input -vhtolerance 0.26 -vltolerance 0.26 -cap 2.29e-15 -quiet
all A
bb_set_output -vddres 8000 -gndres 9000 -cap 4.01e-16 -quiet all Y
.endbb
```

Example of Single UDN Driver Set Quiet

In the following example, `int1` and `int2` will not be analyzed because the driver is set to quiet. In this case there will be no glitch on `int1` or `int2`. Also, `int1` and `int2` will not be valid attackers because the driver is set to quiet.

Figure 1-3 Example Syntax

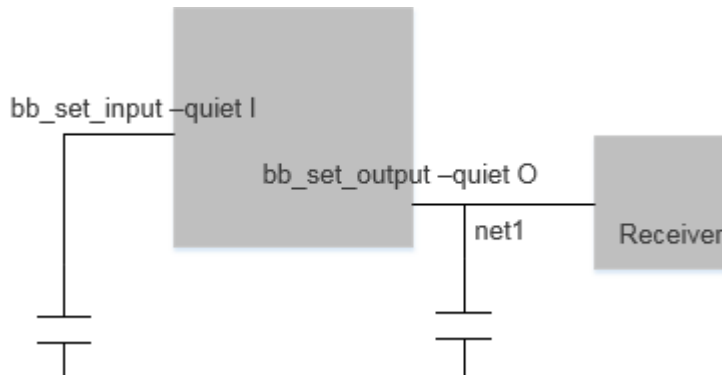
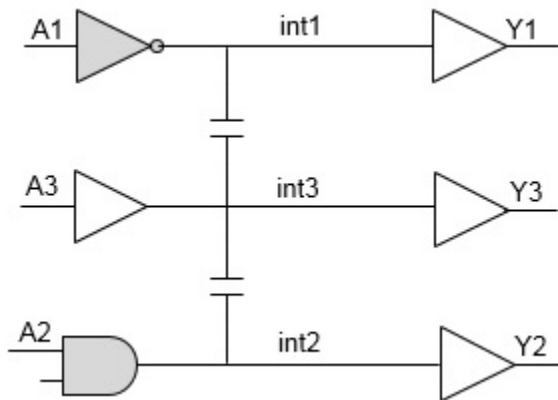


Figure 1-4 Single driver (darkened cells are quiet)



Example of Single UDN Receiver Set Quiet

In the example below, `net2` will be analyzed, but does not show up in the glitch report because the receiver is set to quiet.

Figure 1-5 Example Syntax

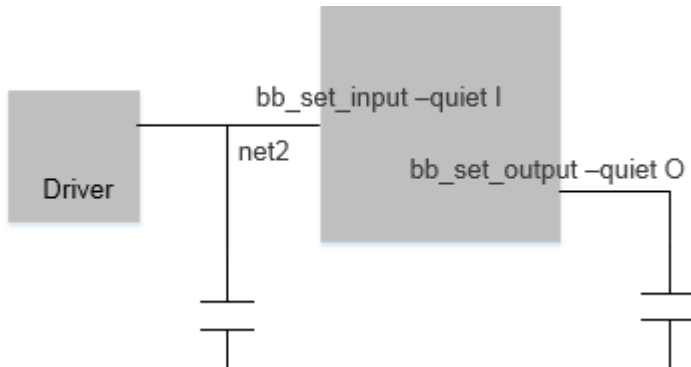
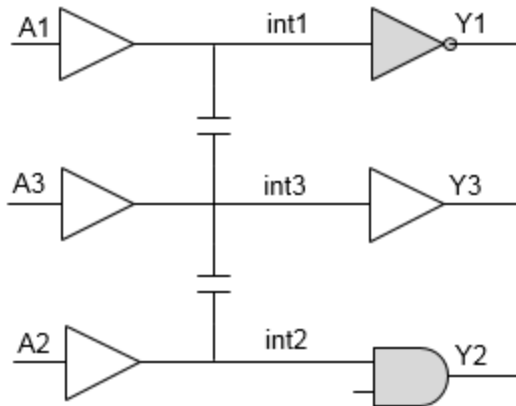


Figure 1-6 Single receiver (darkened cells are quiet)



Example of Multi-Driver with UDN Pin Set Quiet

In the following example, `int1` and `int2` will be analyzed using "Driver" for the driver because the UDN driver pin is set to quiet. The `int1` and `int2` are valid attackers because "Driver" is a valid driving cell.

Figure 1-7 Example Syntax

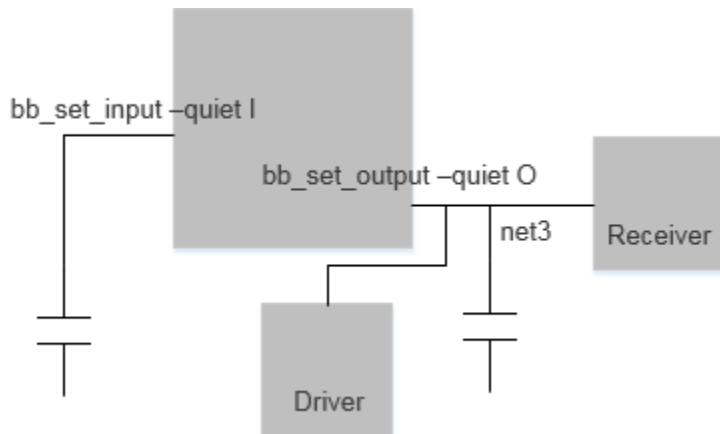
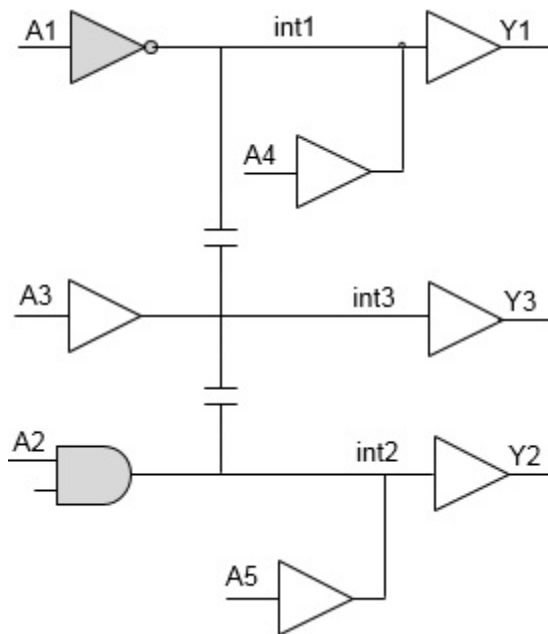


Figure 1-8 Multi-driver (darkened cells are quiet)



Example of Multi-Receiver with UDN Pin Set Quiet

In the example below, int1 and int2 will be analyzed and show up in the glitch report with "Receiver" as the receiving cell; the UDN will not show up in the glitch report. The "Receiver" is the receiving cell with ROP.

Figure 1-9 Example Syntax

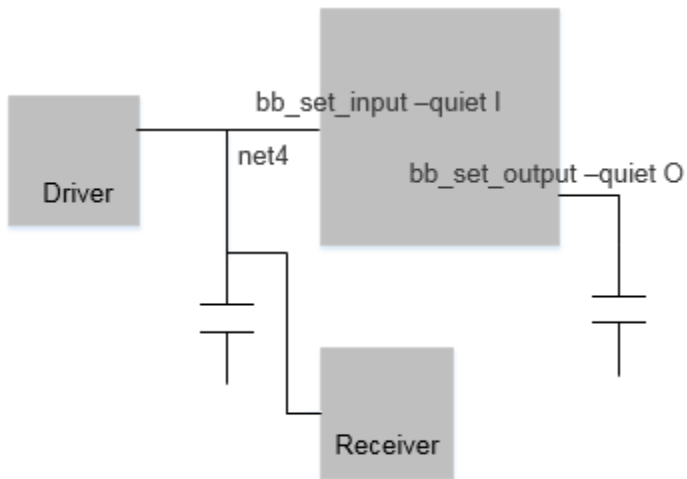
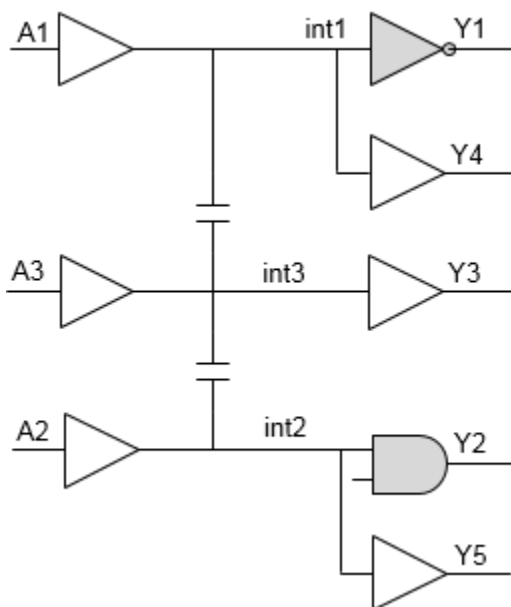


Figure 1-10 Multi receiver (darkened cells are quiet)



Additional Information

- [Slew Propagation](#) on page 30
- [Constructs in Arc Current Models](#) on page 31
- [Characterizing Different Cell Types](#) on page 32
- [Multi-Voltage Characterization](#) on page 39

make_cdB Noise Characterizer User Guide

Noise Library Characterization

- [Generating an ECSM-Noise Liberty Library](#) on page 40
- [Guidelines for Characterizing Large Cells](#) on page 41

Slew Propagation

Note: Slew propagation characterization is not available for UDNs.

Slew propagation requires the following steps:

1. **Characterization:** The signal integrity analyzer characterizes the cell for each input by using a set of slews. This essentially creates a characterization table of data corresponding to each input.
2. **Analysis:** During analysis, the signal integrity analyzer searches the appropriate slew at the driving CCC input from the characterization table. This involves searching through all the possible inputs of the cell for a propagated slew of the appropriate type, and interpolating or extrapolating the data.

Characterization

The signal integrity analyzer feeds in five slews (rise and fall) at each input and determines the propagated slews at all the activated final stage CCC inputs. In situations when a particular final stage CCC input is also an output of the cell, the signal integrity analyzer performs simulations for five different capacitances as well. Each input (rise or fall) slew generates five different slews corresponding to five different capacitances. For each simple CCC input, there are five rise and five fall slews for a cell input pin.

This data is stored in the cdB noise library using the `bb_set_input` command.

For example:

```
bb_set_input -vhtolerance 0.40 -vltolerance 0.40 -cap 2.63e-15 \  
-slews {5.000000e-11 2.000000e-10 3.000000e-10 4.000000e-10 5.000000e-  
10 } \  
-rise_prop_to {A B} \  
-Rise {1.075753e-10 6.988136e-11 1.074155e-10 1.094738e-10 1.074488e-  
10  
-9.152446e-11 -5.473432e-11 -9.191305e-11 -9.811064e-11 -8.939084e-11  
} \  
-fall_prop_to {B} \  
-Fall {-9.153837e-11 -9.373123e-11 -9.117065e-11 -9.196571e-11 -  
9.134613e-11} {I1}
```

make_cdB Noise Characterizer User Guide

Noise Library Characterization

Analysis

During analysis, the signal integrity analyzer picks up the driving CCC input net and the type of slew required (FastRise, FastFall, SlowRise, SlowFall). Then for this net and required slew type, the signal integrity analyzer tries to get all the propagated slew data from the inputs. If there is a requirement for a fast fall slew at the net, the signal integrity analyzer finds the fast input slews at all the inputs. For each of these slews, the signal integrity analyzer tries to find the fall slews at the net using the characterization table. The signal integrity analyzer then uses the minimum values. Slews of any type can be found using the characterization data.

Constructs in Arc Current Models

```
■ bb_vivo_template [-inVdd value] [-outVdd value] [-vi value]
  [-vo value] [-current table]
```

Defines a VIVO template. You must specify this once, but can reuse it multiple times in a cell.

-inVdd value	Specifies the Vdd value at the input.
-outVdd value	Specifies the Vdd value at the output.
-vi value	Specifies the voltage-in value used in characterization.
-vo value	Specifies the voltage-out value used in characterization.
-current table	Specifies the current table.

```
■ bb_index_vector -val {0 0.222 0.4444}_INDEX1
```

Can be used as an index template. Using this template is not a requirement, but using it could save space.

In addition to these constructs, the following set of `bb_set_arc` statements have been added:

```
bb_set_arc [-from arc_input] [-to arc_output] [-dir {in | out | inout}]
  [-type {inv | n-inv}] [-stimuli { {net0 | 1} ...}] [-gndcap value]
  [-millercap value] [-current table]
```

-from arc_input	Specifies from the input of the arc.
-to arc_output	Specifies to the output of the arc.
-dir {in out inout}	Specifies the direction type.
-type {inv n-inv}	Specifies the type as inverting or non-inverting. <i>Default: inv</i>

make_cdB Noise Characterizer User Guide

Noise Library Characterization

<code>-stimuli { {net0 1} ...}</code>	Specifies the list of stimuli during characterization.
<code>-gndcap value</code>	Specifies the loading capacitance value.
<code>-millercap value</code>	Specifies the Miller capacitance value.
<code>-current table</code>	Specifies the list of current tables.

The following example shows what the arc current model information looks like in a cdB noise library file:

```
bb_index_vector -val {0.000000 0.366667 0.733333 1.100000 1.466667 1.833333  
\  
2.200000 2.566667 2.933333 3.300000 } VI_INDEX0  
bb_vivo_template -inVdd -3.3 -outVdd 3.3 -vi VI_INDEX0 -vo VI_INDEX0 \  
-current { 0.0001756 0.0001703 0.000165 ...} inv_VIVO0  
bb_set_arc -arctype min_rise -from in -to out -dir in -type inv \  
-loadcap { 1.6718e-15 } -millercap { 3.11962e-16 } -stimuli { in 1 }  
inv_VIVO0
```

Note: The text version of the cdB noise library file is now about half the size of previous versions.

Characterizing Different Cell Types

The following sections provide information on how to characterize different types of cells:

- [Tiehi and Tielo Cells](#)
- [RAM Cells](#)
- [User-Defined Noise Models](#)
- [Filler/Antenna/Decoupling Cells](#)
- [Sequential and Combinational Cells](#)
- [Level-Shifter Cells](#)

make_cdB Noise Characterizer User Guide

Noise Library Characterization

Tiehi and Tielo Cells

While characterizing tiehi and tielo cells, ensure the following:

Use the default value for port pull-up and pull-down resistance by setting the `port_vdd_r` and `port_gnd_r` variables to 1, respectively.

Specify the tiehi and tielo cells using the `-cell_list` option of the `generate_cell_lib` command.

The following example shows how to characterize tiehi and tielo cells:

```
message_handler -set_msg_level DEFAULT

set_parm port_vdd_r 1
set_parm port_gnd_r 1

set_supply -vdd 1.0 -gnd 0.0
read_dotlib sample.lib
generate_cell_lib -cell_list { TIEHI TIELO } \
                  -vdd { VDD } -gnd { VSS VSS1 } \
                  -file_list { net.SPICE.TT } \
                  -file sample.cdB \
                  -text
```

RAM Cells

The cdB generated by `make_cdB` contains a cell-level and a transistor-level view. The transistor-level view is necessary to propagate the noise through the cdB's and apply propagation based noise filtering. In case of large cells, such as RAM (memory) cells, the transistor-level view contains only the peripheral transistors connected to the pots of the cell. This allows the signal integrity analyzer to propagate noise one level into the cell.

Note: Characterizing large cells, such as RAM cells using `make_cdB` and using the generated cdB for signal integrity analysis, might result in high runtime. If the memory block has less than 100K transistors, `make_cdB` can characterize it in the normal way. However, when the macro size increases, it might be necessary to remove the core of the macro or memory block. This is because `make_cdB` flattens the block before it performs calibration. The flattening process can consume a lot of memory, often more than what is available, resulting in an early termination of calibration.

make_cdB Noise Characterizer User Guide

Noise Library Characterization

To characterize RAM cells, identify the core of the memory and remove the core bits. A typical RAM cell contains six transistors for a bit. The following is an example of a typical RAM cell:

```
.SUBCKT CELL WL BL BLB VDD VSS
XI TT BB VDD INV LN=0.10 WN=0.20 LP=0.15 WP=0.15
MN2 BL WL TT VSS NM L=0.15 W=0.15
MN1 BLB WL BB VSS NM L=0.15 W=0.15
X2 BB TT VDD INV LN=0.10 WN=0.20 LP=0.15 WP=0.15
.ENDS
```

To remove the core bits, you can delete all the lines between `.SUBCKT` and `.ENDS`. Alternatively, you can create an empty `.SUBCKT` for the cell and store it in a separate file, which will be called before the main SPICE file to replace the RAM cell in the netlist. The following is an example of an empty SUBCKT for this cell:

```
*title line
.SUBCKT CELL WL BL BLB VDD VSS
.ENDS
```

When this SUBCKT is properly loaded, all the bit cells are empty. The remaining control logic is characterized by `make_cdb` for use in the signal integrity analysis. The `make_cdb` characterization runtime can be minimized by removing as much of the core of the memory as is possible. All that needs to remain is at least two series cells from the boundary pins of the memory block for proper characterization by `make_cdb`. Assuming that you have created an empty SUBCKT and stored it in a file called `dummy_bit_cell.spi`, the tcl file for `make_cdb` will look as follows:

```
set_parm port_vdd_r 1
set_parm port_gnd_r 1

set_supply -vdd 1.08 -gnd 0.0

#Use the read_dotlib command to extract pin directions from the typical.lib
file.
read_dotlib typical.lib

# Characterize only the necessary cells (top level cell) by
# listing the cells in -cell_list option.
```

make_cdB Noise Characterizer User Guide

Noise Library Characterization

```
# Avoid using the -all option. Only characterize the needed ones.
# List all of the vdds and gnnds.
# Load the dummy_bit_cell.spi first using the -bit_cell_list option.
generate_cell_lib \

    -vdd {VDD} \

    -gnd {VSS} \

    -cell_list ram512_4r4s4 \

    -bit_cell_list { dummy_bit_cell.spi main.spi } \

    -file ram512_4r4s4.cdB
```

Note: `make_cdb` will load the dummy bit cell first, and then it will encounter another subckt by the same name. A warning will be issued that the first definition loaded will be used and the second one will be ignored. This is acceptable, and is the expected behavior.

User-Defined Noise Models

If you have an analog block that has one or more analog pins, `make_cdb` can characterize a user-defined noise model (UDN) for the analog block. This UDN will contain only the cell-level view and there will be no transistor-level view. This cell-level view contains pin capacitance, calibrated input noise threshold, and V_{ds} - I_{ds} curves representing the nonlinear output drive strength. Use the `generate_cell_lib` command with the `-udn_list` option to specify a list of cells in the cell library that should only have UDNs. In the following example, the `my_analog_cell` UDN is specified with the `-udn_list` option:

```
set_parm port_vdd_r 1
set_parm port_gnd_r 1

set_supply -vdd 1.08 -gnd 0.0

#Use the read_dotlib command to extract pin directions from the typical.lib
file.
read_dotlib typical.lib

#Characterize only the necessary cells (top level cell) by listing the cells
```

make_cdB Noise Characterizer User Guide

Noise Library Characterization

```
#in udn_list option.
#All cells in the UDN list must not be included in any other list such as
-cell_list.
#Avoid using the -all option, only characterize the needed ones.
#List all of the vdds and gnds.
#Load the dummy subckts first using the -file_list option.
generate_cell_lib \

    -vdd {VDD} \

    -gnd {VSS} \

    -udn_list { my_analog_cell }\

    -file_list { main.spi } \

    -file analog_SS_1.08V_125C.cdB
```

Filler/Antenna/Decoupling Cells

A filler cell is a cell for which the only thing characterized by `make_cdb` is pin capacitance. Filler cells are also called antenna or decoupling cells. No noise reporting is done for a filler cell. Input pins will have just the capacitance and a very high input noise tolerance. The high input noise tolerance will prevent any failures from being reported for the input pins of this cell. These cells do not have active output drivers and often do not have synthesis library definitions. Declare the port direction on the ports of these cells using the `set_port` command when no synthesis library definition is available. The following example illustrates characterization for the `ANTENNA` filler cell using the `-filler_list` option:

```
set_parm port_vdd_r 1
set_parm port_gnd_r 1

set_port -dir in ANTENNA I
set_supply -vdd 1.2 -gnd 0.0
read_dotlib Antenna.lib
generate_cell_lib \
```

make_cdB Noise Characterizer User Guide

Noise Library Characterization

```
-vdd {VDD} \  
  
-gnd {VSS GND 0 } \  
  
-filler_list { ANTENNA } \  
  
-file_list { main.spi } \  
  
-file ANTENNA_TT_1.2V_25C.cdB
```

Sequential and Combinational Cells

The procedure for characterizing sequential and combinational cells is the same as characterizing any other standard cell. The following example illustrates how to characterize sequential and combinational cells by specifying the DFF1CK_G34 sequential cell and the XORF201_G18 combinational cell with the `-cell_list` option:

```
set_parm port_vdd_r 1  
set_parm port_gnd_r 1  
  
set_supply -vdd 1.08 -gnd 0.0  
read_dotlib typical.lib  
  
generate_cell_lib \  
  
    -vdd {VDD} \  
  
    -gnd {VSS} \  
  
    -cell_list { DFF1CK_G34 XORF201_G18 } \  
  
    -file_list { main.spi } \  
  
    -file out.cdB
```

make_cdB Noise Characterizer User Guide

Noise Library Characterization

Level-Shifter Cells

To characterize a level-shifter cell, use the `set_supply` command to specify the nominal voltage and the `set_net` command to specify the other voltage values.

The following example shows how to characterize a level-shifter cell:

```
# Specify the voltages that will be used to characterize each cell.
set_supply -vdd 1.08 -gnd 0
```

```
# Specify the voltages for all non-nominal supplies.
set_net -vdd 2.0 vdd2v
set_net -vdd 3.0 vdd3v
```

```
# Load the timing library.
read_dotlib level_shifter.lib
```

```
# Generate the noise library for all the cells.
```

```
generate_cell_lib \
-cell_list {low2high high2low} \
-file_list {main.sp} \
-vdd {vdd2v vdd3v vdd} \
-gnd {vss} \
-file level_shifter.cdB \
-text
```

In the above example:

The nominal voltage is 1.08V (for vdd).

There is a 2V supply (vdd2v) and a 3V supply (vdd3v) for the level-shifter as well.

The `main.sp` file contains:

make_cdB Noise Characterizer User Guide

Noise Library Characterization

* top line is always a comment line

```
.inc models.sp
```

```
.inc level_shifter.sp
```

```
.temp 125
```

Multi-Voltage Characterization

To create a cdB file with cells characterized for multiple voltages, use the `set_supply` command to specify each voltage. The following example will characterize each cell with VDD listed at 1.08V, 2.0V, and 3.0V and each cell with VDD25 at 1.5V, 2.5V, and 3.5V.

```
# Specify the supply values for characterization
```

```
set_supply -vdd 1.08 -gnd 0
```

```
set_supply -vdd 2.0 -gnd 0
```

```
set_supply -vdd 3.0 -gnd 0
```

```
set_net -nomvdd 2.5 VDD25
```

```
set_net -nomvdd 3.5 VDD25
```

```
set_net -nomvdd 1.5 VDD25
```

```
# Generate the noise library for each cell.
```

```
# Each cell will be characterized at the above voltages.
```

```
generate_cell_lib \
```

```
-cell_list {
```

```
BUF
```

```
DFF
```

```
NV
```

```
CLKBUF
```

```
LVLSHFT
```

```
} \
```

```
-vdd { VDD VDD25 } \
```

make_cdB Noise Characterizer User Guide

Noise Library Characterization

```
-gnd { VSS } \  
-file_list {  
main.spi  
subckts  
}\  
  
-synlib cell_w.lib \  
-file sample.cdB \  
-text
```

Generating an ECSM-Noise Liberty Library

To generate a Liberty (.lib) library that contains effective current source model (ECSM)-based noise data, use the `-ecsm_si` option of the `generate_cell_lib` command.

Note: Ensure that you specify the input Liberty library using the `read_dotlib` command.

The following example shows how to generate a Liberty file with ECSM-noise extensions:

```
read_dotlib test.lib  
generate_cell_lib \  
    -vdd {vdd vcc} \  
    -gnd {gnd vss} \  
    -cell_list { \  
        inv buf \  
    } \  
    -file_list { \  
        main.sp \  
        inv_chain.sp \  
    } \  
    -file lab1.sp \  
    -ecsm_si
```

The above script generates the following files:

- `make_cdbtest.lib`: Contains the ECSM-noise data.
- `lab1.sp`: Contains the SPICE model and subckt information in encrypted format.

Note: To generate the SPICE model and subckt information in text format, use the `-text` option.

make_cdB Noise Characterizer User Guide

Noise Library Characterization

Note: After you generate the above files, you can load them in Encounter Timing System to perform signal integrity analysis. You must load the library and SPICE files in Encounter Timing System using the following commands:

```
read_lib make_cdbtest.lib  
read_lib -cdb lab1.sp
```

Guidelines for Characterizing Large Cells

- Avoid memory and performance-related issues.

Try characterizing subsets of cells at a time. If all the cdB noise libraries need to be in one file, use the `merge_cdb` command after you have generated the cdB noise libraries for these subsets.

- Avoid putting too much information into the cdB noise library.

For noise analysis, the peripheral transistors of the cell are of primary importance. You can set the `-maxsize` option to a low value and force `make_cdB` to strip off the non-critical transistors of the cell, and keep only the peripheral ones.

- Avoid using UDNs in the signal integrity analyzer whenever possible.

For large cells, such as memories, it is recommended that you black-box the bit cell for cdB characterization in `make_cdB` and then use the resulting cdB for signal integrity analysis. However, for a purely analog block, it is recommended that you use UDNs. For detailed explanation on how to characterize large macros, such as memories, see [RAM Cells](#).

- Avoid processing large channel-connected regions of transistors by setting the following global variables to small values:

- ☐ `ccc_max_ports`
- ☐ `ccc_max_fets`
- ☐ `ccc_max_nets`
- ☐ `large ccc max fets`

make_cdB Commands

- [bb_set_bidi](#) on page 44
- [bb_set_input](#) on page 46
- [bb_set_output](#) on page 48
- [create_ecsm_si_lib](#) on page 49
- [generate_cell_lib](#) on page 50
- [merge_cdb](#) on page 57
- [message_handler](#) on page 58
- [pcdb](#) on page 70
- [print_net](#) on page 72
- [read_dotlib](#) on page 73
- [redirect_output](#) on page 74
- [regenerate_cell_lib](#) on page 75
- [report_library](#) on page 78
- [reset_redirection](#) on page 81
- [set_cell_input_slewlist](#) on page 82
- [set_message_hook](#) on page 83
- [set_net](#) on page 86
- [set_port](#) on page 88
- [set_run_mode](#) on page 90
- [set_simwrap_mode](#) on page 91
- [set_supply](#) on page 92

make_cdB Noise Characterizer User Guide

make_cdB Commands

- set_supply_char_vals on page 93
- validate_cell_lib on page 95

bb_set_bidi

```
bb_set_bidi
  [-vhtolerance float]
  [-vltolerance float]
  [-cap float]
  [-cannotfloat]
  [-vddres float]
  [-gndres float]
  [-vhnoise {time_value_pairs}]
  [-vlnoise {time_value_pairs}] {ports}
```

Marks the specified ports as a bidirectional input/output port. This command has most of the features of the `set_input` and `set_output` commands. The defaults for inputs and outputs are used for bidirectionals.

Parameters

<code>-vhtolerance float</code>	Sets the VH DC input noise margin in Volts on the port. See the <code>output_tolerance</code> global variable. Default: 40 percent of the default VDD/GND swing below nominal VDD.
<code>-vltolerance float</code>	Sets the VL DC input noise margin in Volts on the port. See the <code>output_tolerance</code> global variable. Default: 40 percent of the default VDD/GND swing above nominal ground
<code>-cap float</code>	Sets the capacitance-to-ground value at the port. Units in farads. For example 5e-15. Default: 0 farads
<code>-cannotfloat</code>	Specifies that this port cannot float. Default: Bidirectional ports are allowed to float.
<code>-vddres float</code>	Specifies the RH resistance of the static port model in Ohms. Default: 1000.0
<code>-gndres float</code>	Specifies the RL resistance of the static port model in Ohms. Default: 1000.0

make_cdB Noise Characterizer User Guide

make_cdB Commands

<code>-vhnoise</code> <code>{time_value_pairs</code> <code>}</code>	Specifies a list of time-value pairs representing the VH waveform to be applied at the ports. Values are specified in absolute voltages.
<code>-vlnoise</code> <code>{time_value_pairs</code> <code>}</code>	Specifies a list of time-value pairs representing the VL waveform to be applied at the ports.
<code>{ports}</code>	Specifies a required list of ports to be set as bidirectional.

Example

```
bb_set_bidi adr
```

where net `adr` is set as bidirectional.

bb_set_input

```
bb_set_input  
    [-vhtolerance float]  
    [-vltolerance float]  
    [-cap float]  
    {ports}
```

Marks the specified ports as UDN inputs.

Parameters

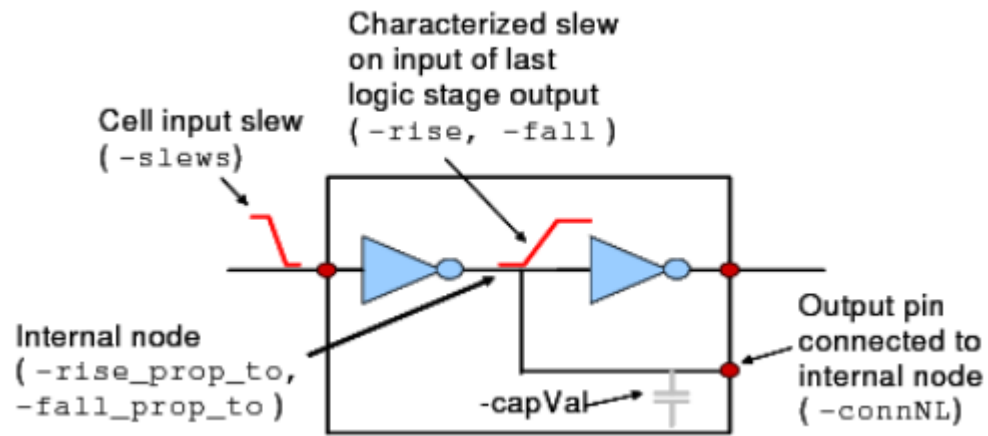
<i>-vhtolerance float</i>	Sets the VH DC input noise margin, in Volts, that this port can tolerate. See the output_tolerance global variable. Default: 40 percent of the default VDD/GND swing below nominal VDD swing.
<i>-vltolerance float</i>	Sets the VL dc input noise margin, in Volts, that this port can tolerate. See the output_tolerance global variable. Default: 40 percent of the default VDD/GND swing above nominal ground voltage
<i>-cap float</i>	Sets the capacitance-to-ground loading to be applied at the port. Units in farads. For example 5e-15. Default: 0 farads
<i>{ports}</i>	Specifies a required list of port names to be set as UDN inputs.

Example

```
bb_set_input {porta portb}
```

where nets, porta and portb, are set as primary inputs.

Figure 2-1 Internal Slew Characterization



bb_set_output

```
bb_set_output
  [-vddres float]
  [-gndres float]
  [-canfloat]
  [-cap float]
  [-vhnoise {time_value_pairs}]
  [-vlnoise {time_value_pairs}] {ports}
```

Marks the specified ports as a UDN output.

Parameters

<code>-vddres float</code>	Specifies the RH resistance of the static port model in Ohms. Default: 1000.0
<code>-gndres float</code>	Specifies the RL resistance of the static port model in Ohms. Default: 1000.0
<code>-canfloat</code>	Specifies that this port is allowed to float. Default: Output ports are not allowed to float
<code>-cap float</code>	Sets the capacitance to ground at the port. Default: 0
<code>-vhnoise {time_value_pairs }</code>	Specifies a list of time-value pairs representing the VH waveform to be applied at the output ports.
<code>-vlnoise {time_value_pairs }</code>	Specifies a list of time-value pairs representing the VL waveform to be applied at the output ports.
<code>{ports}</code>	Specifies a required list of ports to be set as UDN outputs.

Example

```
bb_set_output {porta portb}
```

where the nets, `porta` and `portb`, are set as primary outputs.

create_ecsm_si_lib

```
create_ecsm_si_lib  
    -cdb cdb_name  
    -synlib lib_name
```

Generates a `.lib` file containing ECSM-SI extensions by using an existing `cdB` and a `.lib` file as input.

Parameters

<code>-cdb <i>cdb_name</i></code>	Specifies the name of the input <code>cdB</code> noise library.
<code>-synlib <i>lib_name</i></code>	Specifies the name of the input <code>.lib</code> file.

Example

- The following command creates an ECSM-SI `.lib` named `my_ecsm_si_typical.lib` by using the `base.cdB` and `typical.lib` files as input:

```
set_parm filename_prefix my_ecsm_si_  
create_ecsm_si_lib -cdb base.cdB -synlib typical.lib
```

generate_cell_lib

```
generate_cell_lib
  [-all]
  [-bit_cell_list bit_cell_list]
  [-ccc_print_large value]
  [-cell_list {list}]
  [-ecsm_si]
  [-file_list {list}]
  [-filler_list {list_of_cells}]
  [-gnd {netnames}]
  [-maxsize {integer}]
  [-skip {list}]
  [-skip_failed_cells]
  [-spectre_file_list {list_of_spectre_files}]
  [-strip_supply]
  [-text]
  [-tolerance]
  [-udn_list {list}]
  [-vdd {netnames}
```

Generates the noise library that can be used to represent the cells while performing signal integrity analysis. The `set_supply`, `set_net`, and `set_port` commands (for unbuffered cells) must be run before the `generate_cell_lib` command for correct noise characterization.

Important

It is mandatory to specify the `set_supply` command before the `generate_cell_lib` command.

This command is supported by the `make_cdB` utility, a noise characterization tool that encapsulates the essential information inside a cell for noise analysis purposes. Resistances, capacitances, and noise tolerance characteristics for each cell are taken into account. The characterized cdB noise library usually has a `.cdB` extension.

Ensure that you use the `read_dotlib` command to specify a timing library file for port direction settings before using this command. In addition, specify all the required `make_cdB` global variables and other Tcl commands before issuing this command.

This command should be issued only once in each call to the `make_cdB` utility.

make_cdB Noise Characterizer User Guide

make_cdB Commands

Parameters

<code>-all</code>	Characterizes all unique subcircuits in the SPICE files specified in <code>-file_list</code>. This option can lead to longer run times because library cells are often built from smaller component cells that are characterized. It also can cause errors when <code>make_cdB</code> attempts to calibrate subcircuits that are not placeable and do not have valid default parameters, such as L and W. Do not use this option with the <code>-cell_list</code> option. Important: Option <code>-all</code> is obsolete and has been replaced by the <code>-cell_list</code> option. Although the <code>-all</code> option will continue to work, the software will issue a warning message indicating that this option is obsolete. To avoid the warning message, update your script to replace the <code>-all</code> option with the <code>-cell_list</code> option.
<code>-bit_cell_list</code> <i>bit_cell_list</i>	Specifies a list of bit cells present in the memory. When this option is used, <code>make_cdB</code> automatically creates the empty black-boxes for the subcircuits of the specified bit cells by ignoring any internal devices.
<code>-ccc_print_large value</code>	Reports the names of all channel-connected components (CCCs) that have more than the specified number of transistors.
<code>-cell_list {list}</code>	Specifies a required list of cells in the cell library that are characterized for noise.
<code>-ecsm_si</code>	Generates a Liberty (.lib) library that contains effective current source model (ECSM)-based noise data. Before you use this option, you must load a Liberty (.lib) library using the <code>read_dotlib</code> command. Note: The output Liberty library, which contains ECSM-noise data, can be used as input to Encounter Timing System for performing noise analysis.
<code>-file string</code>	Specifies the required output noise library file name.

make_cdB Noise Characterizer User Guide

make_cdB Commands

`-file_list {list}`

Specifies a required list of SPICE files that contain the transistor-level netlist of the cells and SPICE model files. If the SPICE models are specified using the library (`.lib`) format, they cannot be directly loaded. Instead, create a main SPICE file and load this file.

Note: Use this option to load an ECHO file generated using a transistor-level signal integrity analysis tool.

Important: The cdB noise library is calibrated at the temperature specified by the `.temp` command in the SPICE format input files. If the temperature is not specified in the SPICE files, then the default temperature specified in the BSIM SPICE models is used during calibration. You must provide a comment line as the first line in the SPICE file (in title at the top of the file) because the signal integrity analyzer and the `make_cdB` SPICE netlist parser ignore the first line in the SPICE file.

`-filler_list
{list_of_cells}`

Specifies a list of filler cells that need to be characterized.

A filler cell is a cell that the `make_cdB` utility characterizes only for capacitance and for which no noise reporting is done.

Note: Do not specify filler cells using the `-cell_list` option.

`-gnd {netnames}`

Specifies a list of net names to be treated as ground.

Any supply name that contains a colon delimiter and a suffix will be treated as a supply if the prefix to the colon is specified in the `-gnd` list. For example, `gnd:3`.

Note: There are no default supply net names. The circuit supply nets must be specified.

make_cdB Noise Characterizer User Guide

make_cdB Commands

<code>-maxsize {integer}</code>	Specifies that cells with a transistor count above this limit are characterized for gate level, but have a minimized transistor-level representation. Logical information does not propagate through this cell. This limit is not applied to cells specified in the <code>-udn_list</code>. This limit also helps to prevent excessive run time and memory usage when the <code>-all</code> option is used. <i>Default:</i> 100 Note: When the number of transistors in a cell exceeds the <code>-maxsize</code>, the <code>make_cdB</code> utility issues a warning and then creates an ECHO for the transistor-level representation of the cell. The maximum legal value for the <code>-maxsize</code> option is 2000. This limitation helps to avoid excessively large noise libraries. When an ECHO is utilized for the transistor-level view of a cell, the signal integrity analyzer is not able to propagate noise through that cell.
<code>-skip {list}</code>	Specifies a list of cells that are not calibrated. This command is particularly useful when used in combination with, but is not limited to, the <code>-all</code> option.
<code>-skip_failed_cells</code>	Causes all cells that have failed calibration on any pin to be excluded from the noise library. <i>Default:</i> Include all cells in the noise library.

<code>-spectre_file_list</code> <code>{list_of_spectre_files}</code>	Specifies a list of valid Spectre[®] files. You can combine both SPICE and Spectre netlists as part of the Spectre netlist. The Spectre simulator includes support for the SPICE netlist from the Spectre netlist. For example, a top-level Spectre netlist can include both SPICE and Spectre in a single file:
---	--

Top-level Spectre Netlist File:

```
simulator lang=spectre
include "model_pt.spe"
```

```
simulator lang=spice
.include inv_chain.spi
```

Note: Make sure to include the `simulator lang=spectre` line at the beginning of each Spectre netlist.

After you set up the Spectre file netlist, you can use it as follows:

```
generate_cell_lib -spectre_file_list
{top.scs}
```

CMI Requirements

The signal integrity analyzer supports the standard elements of Spectre and the models that are dynamically loaded by Spectre using the compiled model interface (CMI) shared object feature.

If you are reading Spectre files that contain CMI elements, this option requires specifying the locations of the shared object files that contain the descriptions of the additional elements.

To specify the locations of these shared object files, use the `cmiconfig` utility described in Chapter 4, Syntax, Configuring CMI Shared Objects in the *Spectre® Circuit Simulator Reference*, which is shipped with the MMSIM release.

For example:

```
setenv CMI_CONFIG /path1/user_cmiconfig
where user_cmiconfig contains instructions to load
the CMI libusermodel.so file, such as:
load /path2/libusermodel.so
```

Note: The `.so` DLL files are platform specific and must be compiled for the platform that is being run.

All nets specified as `-vdd` and `-gnd` should have a matching global declaration in the Spectre files.

This option requires that you have Spectre included in your PATH. The generation of temporary files in the current directory is minimal.

`-strip_supply`

Drops power supply ports from the port list of each cell. During signal integrity analysis, the cell ports are stored in the noise library.

`-text`

Requests a text output format. The output file is named using a `.txt` extension. The text output is only created if all input files are text. For example, if the signal integrity analyzer reads in an encrypted cdB noise library, all cdB noise libraries and ECHO models are encrypted.

Default: Without this option, only a binary version is output.

make_cdB Noise Characterizer User Guide

make_cdB Commands

<code>-tolerance</code>	Activates cell-level DC tolerance. After this option is set, the V_H or V_L DC tolerance of a cell can be changed by using the <code>port_vh_noise_tolerance</code> or <code>port_vl_noise_tolerance</code> global variables. Note: By default, make_cdB automatically computes the V_L or V_H tolerance. Using the <code>-tolerance</code> option enables you to override the automatic calculation using the <code>port_vl_noise_tolerance</code> or <code>port_vh_noise_tolerance</code> global variables, which is not recommended.
<code>-udn_list {list}</code>	Specifies a list of cells in the cell library that are converted into a user-defined noise (UDN) model. The cells in this list are calibrated for gate-level analysis only. They do not contain transistor-level information for use in delay and sensitivity analysis. All cells in the UDN list must not be included in any other list such as <code>-cell_list</code>.
<code>-vdd {netnames}</code>	Specifies a list of net names to be treated as supply. Any supply name that contains a colon delimiter and a suffix will be treated as a supply if the prefix to the colon is specified in the <code>-vdd</code> list. For example, <code>vdd:3</code>. Note: There are no default supply net names. The circuit supply nets must be specified.

Examples

- The following command characterizes the XOR cells in the cell library for noise, loads the `main.spi` SPICE file that contains the transistor-level netlist and SPICE models, and outputs the noise library file as `cells.cdB`:

```
read_dotlib timing.lib
generate_cell_lib -cell_list { XOR } -file_list { main.spi } -file cells.cdB
```

where `main.spi` contains:

```
* title
.lib model_lib.spi tt
.include cells.spi
```


merge_cdb

```
merge_cdb
  [-cdb {list}]
  [-no_unique]
  [-text]
  -file filename
```

Concatenates multiple noise library files into a single noise library file. If duplicate `.bbox` or `.subckt` definitions are found, the first definition is kept and all others are ignored. By default, this command outputs the merged cdB in encrypted format.

Parameters

<code>-cdb {list}</code>	Specifies the names of the cdB noise libraries to concatenate.
<code>-file filename</code>	Specifies the required output file name.
<code>-no_unique</code>	Allows all <code>.bbox</code> and <code>.subckt</code> definitions to be kept, even if duplicated.
<code>-text</code>	Requests a text output format for the merged cdB. All input cdBs must be in text format to use this option. Note: When this option is specified, make_cdB generates the merged cdB in text as well as encrypted format.
<code>-uniq_models</code>	Retains only unique transistor models (<code>.model</code> definitions) in the merged cdB. When you specify this option, the software detects duplicate models in the input cdB files and retains only one definition for such models in the resultant merged cdB. Even if the model names are different the software detects the identical models based on the parameter values, and retains only one definition.

Examples

- The following command concatenates the `stdcel.cdb` and `custom.cdb` noise libraries into a single cdB noise library named `main.cdb`.

```
merge_cdb -cdb {stdcel.cdb custom.cdb} -file main.cdb
```

- The following command merges two cdB files and generates a cdB file without duplicate model definitions:

```
merge_cdb -cdb { lab1.cdb lab2.cdb } -file test_merge -text -uniq_models
```

make_cdB Noise Characterizer User Guide

make_cdB Commands

message_handler

```
message_handler
  [-msg_count_summary {OFF | ON}]
  [-no_format]
  [-revive]
  [-set_constraints {{-id message_id_list | -level message_level |
  -context ctxt_list -thislevelonly} | -constraint {0 | positive_integer |
  -1 | msg_id_limit}}]
  [-set_error_limit number]
  [-set_info_limit number]
  [-set_msg_level {ERROR | DEFAULT | VERBOSE | ALL | msg_level}]
  [-set_output_policy {-file filename | -discard_file filename | -gzip |
  -detail {terse | normal | high}}]
  [-set_screen_display {OFF | ON}]
  [-set_warn_limit number]
  [-show_constraints { -id | -global | -context | -level | -all}]
  [-show_current_msg_level]
  [-show_msg_level message_id]
  [-show_report {-id {message_id_list | ALL} | -context {message_ctxt_list |
  ALL}}]
  [-suspend]
```

Lets you control the detail and the types of messages that make_cdB displays. A message contains the message severity level, message ID, and message content. For example:

```
INFO:(SI-6050) Now using /tmp/make_cdb_linws9.Cadence.COM_28215 (32361 MB
available)
```

where:

INFO:	Specifies the message severity level. make_cdB supports three message severity levels: ERROR, WARNING, and INFO. Each message severity level is associated with a message level. Note: make_cdB does not print the message level with the message. To get the message level for a message, use the following command: <code>message_handler -show_msg_level <i>message_id</i></code>
SI-6050	Specifies the message ID.
Now using...	Specifies the message content.

make_cdB Noise Characterizer User Guide

make_cdB Commands

Parameters

<code>-msg_count_summary</code> {OFF ON}	Outputs statistics on the number and types (message levels) of messages make_cdB encounters during the run. make_cdB provides these statistics at the end of a run, when the message handler is closed by the application, or when you use the <code>-show_report</code> option. Selecting the OFF argument will not provide statistics at the end of the run. <i>Default:</i> OFF
<code>-no_format</code>	Turns off message formatting to keep messages from wrapping at the 80-column width limit. Each message will be displayed on one line. This option is useful when you <code>grep</code> for specific message information that is displayed on multiple lines.
<code>-revive</code>	Revives the message handler by updating the message settings.
<code>-set_constraints {-id message_id_list -level message_level -context ctxt_list -thislevelonly}</code> <code>-constraint {0 positive_integer -1 msg_id_limit}</code> <code>-id message_id_list</code>	Controls the number of messages that make_cdB displays based on the message ID and the constraint that is set. You can only use this option if you know the message IDs of the messages you want to display. You can get the message IDs from the messages generated by make_cdB. Note: You can also use this option to either suppress or display non-SI messages, such as TECHLIB, CTE, and FNP. For example, to suppress all the messages with the TECHLIB-342 ID, use the following command: <code>message_handler -set_constraints { -id TECHLIB-342 -constraint 0 }</code>

make_cdB Noise Characterizer User Guide

make_cdB Commands

When controlling the display of non-SI message IDs, it is not possible to set a number for the total messages the software will suppress or display. All the messages with the specified ID will either be suppressed (`-constraint 0`) or displayed (`-constraint -1`).

`-level message_level`

Controls the number of messages that make_cdB displays based on the message level and the constraint that is set. What *message_level* you use is dependent on what messages you want make_cdB to display. To find the message level for a message, use the `-show_msg_level` option.

`-context ctxt_list [-thislevelonly]`

Controls the number of messages that make_cdB generates based on the message context and the constraint that is set. The

`-thislevelonly` option is used exclusively with the `-context` option and applies the constraint only to messages of the specified context type and its message level.

Omitting this option applies the constraint to all messages of the specified context type regardless of its message level. To find the message level for a message, use the `-show_msg_level` option.

`-constraint {0 | positive_integer | -1} | msg_id_limit`

Specifies a constraint for an individual message ID type or a global constraint for all message ID types. These constraints limit the total number of messages that make_cdB displays.

`{0 | positive_integer | -1}`

make_cdB Noise Characterizer User Guide

make_cdB Commands

Sets a constraint on an individual message ID type that is specified with the `-id`, `-level`, or `-context` options.

0: Use this option to discard messages of a particular type.

positive_integer: Use this option to output a specified number of messages.

-1: Use this option to output all messages.

Note: If a constraint is set, and the message ID, message level, and message context all match a specific message, make_cdB applies the constraint in the following order of precedence: message ID, message level, global constraints, and context constraints.

msg_id_limit

Sets a global constraint to limit the total number of messages that are displayed for each message ID type. make_cdB stops displaying messages of the same message ID type if the number of messages are above the specified limit. For example:

```
-set_constraints { -constraint 300 }
```

When a message of a specific message ID type reaches a count of 300, that message ID type will no longer be displayed.

make_cdB Noise Characterizer User Guide

make_cdB Commands

Note: The `msg_id_limit` option provides a global constraint to all message ID types, with the following exceptions:

When using the `msg_id_limit` option in combination with setting a constraint for an individual message ID type, such as:

```
-set_constraints { -id SI-xxxx -  
constraint y }
```

The constraint for the individual message ID type will take precedence.

Note: The individual message ID takes the highest precedence followed by level, global constraints, and context constraints.

When using the `msg_id_limit` option to set a global constraint for all message ID types, the following message ID types will remain unconstrained unless they are specifically constrained using the

`-set_constraints -id` option:

SI-9513

SI-9514

SI-500

SI-100

SI-6000

SI-2000

make_cdB Noise Characterizer User Guide

make_cdB Commands

`-set_error_limit number`

Sets a limit to the number of ERROR messages that make_cdB displays. You can set this *number* to any value. If you set this *number* to 0, make_cdB will not display any error messages. If you set this *number* to -1, make_cdB displays all error messages.

You can use this option multiple times. Each time it is used make_cdB overwrites the previous limit. For example, if you do not want error messages to appear when running the `set_supply` and `set_net` commands, but want all error messages to appear after running the `set_net` command, use this option as follows:

```
message_handler -set_error_limit 0
set_supply...
set_net...
message_handler -set_error_limit -1
...
```

`-set_info_limit number`

Sets a limit to the number of INFO messages that make_cdB displays. You can set this *number* to any value. If you set this *number* to 0, make_cdB will not display any info messages. If you set this *number* to -1, make_cdB displays all info messages.

You can use this option multiple times. Each time it is used make_cdB overwrites the previous limit. For example, if you do not want info messages to appear when running the `set_supply` and `set_net` commands, but want all info messages to appear after running the `set_net` command, use this option as follows:

```
message_handler -set_info_limit 0
set_supply...
set_net...
message_handler -set_info_limit -1
...
```

`-set_msg_level {ERROR | DEFAULT | VERBOSE | ALL | msg_level}`

make_cdB Noise Characterizer User Guide

make_cdB Commands

Sets the types of messages that make_cdB displays.

Default: ERROR

ERROR

Displays all ERROR messages generated by make_cdB.

Message level: 39

DEFAULT

Displays all ERROR messages, important WARNING messages, and important INFO messages generated by make_cdB.

Message level: 49

VERBOSE

Displays all ERROR messages, all WARNING messages, and important INFO messages generated by make_cdB.

Message level: 58

ALL

Displays all ERROR messages, all WARNING messages, and all INFO messages generated by make_cdB.

Message level: 255

msg_level

Specifies a message level from 10 to 255. make_cdB will not display messages that have a message level above this specified *msg_level*. Message level 58 is a typical message level setting. This displays all ERROR messages, all WARNING messages, and important INFO messages.

Table 2-1 on page 62 below describes the message_handler message levels:

```
-set_output_policy {-file filename | -discard_file filename |  
-gzip | -detail {terse | normal | high}}
```


make_cdB Noise Characterizer User Guide

make_cdB Commands

`-gzip | -detail {terse | normal | high}}`

Controls how you want make_cdB to output messages. Use this option to specify a global policy that you want applied to messages.

`-file filename`

Stores the messages in the specified file.

`-discard_file filename`

Stores the messages that are not displayed as specified by the `-set_msg_level` or `-set_constraints` options. make_cdB stores the messages in a file specified by *filename*. If this argument is not specified, make_cdB discards those messages.

`-gzip`

Writes the log file and discard file in gzip format.

`-detail {terse | normal | high}`

Determines the detail of the messages shown. Selecting the `high` argument tells make_cdB to search the message file for messages with a high level of detail.

If the detailed message format is not found, make_cdB uses the `normal` format.

Default: `normal`

`-set_screen_display {OFF | ON}`

Turns the screen display on or off. If you turn the screen display off, messages with a level of 0 and messages with a level between 250 and 255 will only be displayed.

Default: `ON`

`-set_warn_limit number`

make_cdB Noise Characterizer User Guide

make_cdB Commands

Sets a limit to the number of WARNING messages that make_cdB displays. You can set this *number* to any value. If you set this *number* to 0, make_cdB will not display any warning messages. If you set this *number* to -1, make_cdB displays all warning messages.

You can use this option multiple times. Each time it is used, make_cdB overwrites the previous limit. For example, if you do not want warning messages to appear when running the `set_supply` and `set_net` commands, but want all warning messages to appear after running the `set_net` command, use this option as follows:

```
message_handler -set_warn_limit 0
set_supply...
set_net...
message_handler -set_warn_limit -1
...
```

`-show_constraints { -id|-global|-context|-level|-all }`

Displays the constraints of the specified type or all constraints. This option accepts the following arguments:

`-id`: Displays the message IDs and the constraints set for them.

`-global`: Displays the global constraint.

`-context`: Displays the message contexts and the constraints set for them.

`-level`: Displays the message levels and the constraints set for them.

`-all`: Displays all constraints regardless of the type.

`-show_current_msg_level` Displays the current message level that is set.

`-show_msg_level message_id`

make_cdB Noise Characterizer User Guide

make_cdB Commands

Provides the message level of the message specified by *message_id*. For example:

```
message_handler -show_msg_level  
CADSI_WARN
```

where `make_cdB` provides the message level of the message that contains the `CADSI_WARN` message ID.

Note: You can only use this option if you know the message ID of the message. You can get the message ID from the messages generated by `make_cdB`.

```
-show_report {-id {message_id_list | ALL} | -context  
{message_ctxt_list | ALL}}
```

Shows a report on all messages generated by the software that match the `message_id_list` or the `message_ctxt_list`. If the `ALL` keyword is specified, all message IDs or contexts are generated.

```
-id {message_id_list | ALL}
```

Calculates and shows the number of messages with the specified message IDs. When the `ALL` keyword is specified, the software reports the total number of messages generated for all message IDs.

```
-context {message_ctxt_list | ALL}
```

Calculates and shows the number of messages with the specified message context. When the `ALL` keyword is specified, the software reports the total number of messages generated for all message contexts.

Note: The outputs generated by `-id` and `-context` are independent of each other even though it is possible to use them together with a single `-show_report` option.

```
-suspend
```

Suspends all messages, but `make_cdB` updates the number of messages, message levels, and message contexts for each message.

Table 2-1 Message Levels and their Description

Message Handler msg_level	Description
0-10	Specifies the message levels assigned to generic messages. make_cdB displays these messages by default.
11-39	Specifies the message levels assigned to error messages prefixed with the <code>ERROR</code> severity level. When specifying a <code>msg_level</code> in this range, make_cdB displays all error messages that have a message level at or below the specified <code>msg_level</code> .
40-45	Specifies the message levels assigned to important warning messages prefixed with the <code>WARNING</code> severity level. When specifying a <code>msg_level</code> in this range, make_cdB displays all important warning messages that have a message level at or below the specified <code>msg_level</code> . This includes all error messages since they have message levels below the specified <code>msg_level</code> .
46-49	Specifies the message levels assigned to important information messages prefixed with the <code>INFO</code> severity level. When specifying a <code>msg_level</code> in this range, make_cdB displays all important information messages that have a message level at or below the specified <code>msg_level</code> . This includes all important warning messages and all error messages since they have message levels below the specified <code>msg_level</code> .
50-58	Specifies the message levels assigned to warning messages prefixed with the <code>WARNING</code> severity level. When specifying a <code>msg_level</code> in this range, make_cdB displays all warning messages that have a message level at or below the specified <code>msg_level</code> . This includes all important information messages, all important warning messages, and all error messages since they have message levels below the specified <code>msg_level</code> .
59-150	Specifies the message levels assigned to information messages prefixed with the <code>INFO</code> severity level. When specifying a <code>msg_level</code> in this range, make_cdB displays all information messages that have a message level at or below the specified <code>msg_level</code> . This includes all warning messages, all important information messages, and all error messages since they have message levels below the specified <code>msg_level</code> .
151-255	Not used at this time.

Examples

- The following command outputs statistics on the number and type of messages make_cdB encounters during the run.

```
message_handler -msg_count_summary ON
```

- The following commands limit the number messages you see to a specific message ID type. Using these commands greatly reduces the log file size.

```
message_handler -set_constraint { -id ID1 -constraint 5 }  
message_handler -set_constraint { -id ID2 -constraint 0 }  
message_handler -set_constraint { -id ID3 -constraint -1 }
```

where the messages having ID1 appear only five times, messages having ID2 do not appear at all, and messages having ID3 always appear.

- The following command limits the number of error messages to 50.

```
message_handler -set_err_limit 50
```

- The following command stores messages in a file named `run.log`.

```
message_handler -set_output_policy {-file ./run.log}
```

- The following command displays the message IDs and the constraint set for them:

```
message_handler -show_constraints { -id }
```

- The following command reports the total number of messages generated with the ID, SI-6050:

```
message_handler -show_report {-id SI-6050}
```

- The following command reports the total number of messages generated for all message IDs and the message context, make_cdb:

```
message_handler -show_report {-id ALL -context make_cdb}
```

pcdb

```
pcdb
  [-all]
  [-cell_list {list}]
  [-no_ports]
  [-pin_cap]
  [-tolerance]
  [-version]
  -cdb {list}
```

Prints information about cells and user-defined noise (UDN) models inside cdB noise libraries. make_cdB prints the following information:

- cdB noise library version
- Spectre release version, if using the SIMI interface
- Tool release version for each cdB noise library and cell or UDN name
- Temperature used to characterize the cell or UDN
- Port list with port direction (if available)
- Supply (VDD) and ground (GND) net names with voltage values
- VH and VL tolerance values

Parameters

-all	Prints all cells and UDNs in the noise libraries.
-cdb {list}	Specifies a required list of cdB noise libraries from which cell and UDN information is extracted.
-cell_list {list}	Reports information about the list of cells. Note: If you use the -cell_list option with the -all option, the -all option will be ignored.
-no_ports	Outputs only cell and UDN names. No port names are output. Note: When you use the -no_ports option, the -pin_cap option is ignored and the voltage information is not displayed.

make_cdB Noise Characterizer User Guide

make_cdB Commands

<code>-pin_cap</code>	Prints pin capacitance values in the noise library. Note: If the <code>-no_ports</code> option is specified, the <code>-pin_cap</code> option is ignored.
<code>-tolerance</code>	Prints the VH and VL tolerance values of the input cdB. This is particularly useful when the input cdB is encrypted and you want to determine the tolerance values.
<code>-version</code>	Prints the version of make_cdB used to create the cdb.

Examples

- The following command extracts cell and UDN information from the cdB noise library (`lab1.cdB`) and prints the cell or UDN name, cdB noise library version number, temperature, port list with port directions, port capacitance, and voltage for the `inv` list of cells:

```
pcdb -cell_list { inv } -cdb { lab1.cdB } -pin_cap
FILE NAME          CELL NAME          PORT SEQUENCE, PORT CAPACITANCE,VOLTAGE
-----
lab1.cdB
VERSION 1
RELEASE UNKNOWN
TEMPERATURE 25
(This file has model info)
    1. inv (cell)
        1. in (INPUT), 1.28e-15
        2. out (OUTPUT), 1.77e-15, VDD 3.3 gnd 0
```

print_net

```
print_net  
    -cell  
    {net_name}
```

Prints dangling nets.

Parameters

<code>-cell</code>	Specifies the cell name.
<code>net_name</code>	Specifies the net names.

Examples

- The following command prints dangling nets for the BUF cell:

```
print_net -cell BUF {A2_65}
```


read_dotlib

```
read_dotlib  
    dotlib_library_filename | { list_of_dotlib_filenames }
```

Reads the timing library files and extracts pin directions.

Parameters

<i>dotlib_library_filename</i>	Specifies the name of the .lib library. This name can be a fully qualified file name or a relative file name.
<i>list_of_dotlib_filenames</i>	Specifies multiple dotlib library filenames. Use curly braces to specify multiple library filenames, for example: {lib1 lib2 lib3}.

Example

- The following command reads the `example.lib` timing library file:

```
read_dotlib example.lib
```

redirect_output

```
redirect_output  
    log_file_name
```

Redirects the resulting outputs from a set of make_cdB commands to a log file that you specify.

Parameters

<i>log_file_name</i>	Specifies the name of the log file being used for redirection.
----------------------	--

Example

- The following command redirects the output from a set of make_cdB commands to the *log_file_name* that you specify:

```
redirect_output log_file_name  
command01  
command02  
...  
reset_redirection
```

Note: Use the reset_redirection command to nullify the effects of the redirection.

regenerate_cell_lib

```
regenerate_cell_lib  
    [-infile string]  
    [-outfile string]  
    [-temperature]  
    [-text]  
    [-version_string cdB_version]
```

Converts an old cdB noise library file to the latest cdB noise library file version.

Important

It is mandatory to specify the set_supply command before the `regenerate_cell_lib` command.

The `regenerate_cell_lib` command supports multi-voltage cdB regeneration. This means that you can regenerate an existing cdB to create a new cdB with cells characterized at different voltage values specified during regeneration.

See *Regenerating the cdB Noise Library* in the *make_cdB Noise Characterizer User Guide* for more information.

Note: Use the read_dotlib command to specify a timing library file for port direction settings before using this command. By default, `make_cdB` uses the port direction settings from the old cdB noise library file.

Important

This command should be issued only once in each call to the `make_cdB` utility.

Note: The `regenerate_cell_lib` command does not work with SIMI.

Parameters

<code>-infile <i>string</i></code>	Specifies the required file name or location of the old cdB noise library file.
------------------------------------	---

make_cdB Noise Characterizer User Guide

make_cdB Commands

<code>-outfile <i>string</i></code>	Specifies the output file name or location of the new cdB noise library file. Default: Adds file name <code>\${input_cdb}.new</code> to the directory of the old cdB noise library file. If this file name exists, file name <code>\${input_cdb}.new.1</code> is used or the suffix is incremented until a valid file name is found.
<code>-temperature</code>	Specifies the temperature for the regenerated cdB noise library file. If not specified, the temperature value of the old cdB noise library file is used.
<code>-text</code>	Generates a text version of the new cdB noise library file using the output file name with a <code>.txt</code> extension. Note: If the original cdB noise library file that your converting is encoded or not in text format, no text output is generated.
<code>-version_string <i>string</i></code>	Provides a tag to identify the cdB noise library file versions you are using. This is useful in situations where you generate several versions of the cdB noise library file for the same cells, such as for different voltages, process corners, and so on. You can view the version strings using the <code>-version</code> option of the <code>pcdb</code> command.

Examples

- The following commands convert a cdB noise library file from version 4.0 to the latest version, which is identified as `rechar-to-4.3.2`:

#Use `set_parm` as shown, when `tiehi` and `tielo` cells are present.

```
set_parm port_vdd_r 1
```

```
set_parm port_gnd_r 1
```

```
set_supply -vdd 1.8 -gnd 0
```

```
set_net -vdd 1.5 { vdda }
```

```
regenerate_cell_lib -infile old.cdB \  
                    -outfile new.cdB \  
                    -version_string rechar-to-4.3.2
```

make_cdB Noise Characterizer User Guide

make_cdB Commands

- The following commands recharacterize an existing text-based cdB file at multiple voltages:

```
set_run_mode 12
set_parm bind_supply_to_port 1
```

```
set_supply -vdd 3.3 -gnd 0.0
set_supply -vdd 1.7 -gnd 0.0
set_net -vdd 2.0 { VDD2 }
set_net -nomvdd 3.3 -vdd 3.3 VDD
set_net -nomvdd 1.7 -vdd 1.7 VDD
```

```
set_port -dir in TOP {IN1}
set_port -dir out TOP {OUT1}
set_port -dir out TOP {OUT2}
set_port -dir out TOP {OUT3}
```

```
regenerate_cell_lib -infile test.cdB.txt -text
```

report_library

```
report_library
  [-library library_name]
  [-cells [pattern]]
  [-wireloads]
  [-operating_conditions]
  [-pvt {min | typ | max}]
  [-tcl_list]
  [{> | >>} filename]
```

Generates a report about the technology library used in the design.

Parameters

<code>{> >>} <i>filename</i></code>	Specifies the name of the file in which the report is saved. The <i>filename</i> argument must be the last argument in the list. If the file name is not specified, the report is displayed on the standard output without being saved. Since the report is often large, text may be lost if you do not save the report to a file.
<code>-cells [<i>pattern</i>]</code>	Reports information about all cells in the library. If a <i>pattern</i> is specified, only the cell names in the library that match the pattern are reported.
<code>-library <i>library_name</i></code>	Reports contents of the specified library, including non-target libraries.
<code>-operating_conditions</code>	Reports information on all the operating conditions in the library. If none of the <code>-cells</code> , <code>-wireloads</code> , and <code>-operating_conditions</code> options are specified, the report contains information about all the cells, wire-load models, and operating conditions.

make_cdB Noise Characterizer User Guide

make_cdB Commands

<code>-pvt {min typ max}</code>	Reports library data for a particular PVT (process, voltage, temperature) corner. Specify only one PVT corner per command. <i>Default:</i> The library data is reported for the <code>max</code> PVT corner only. If the library is a merged library, the order of reporting is <code>max</code> if it exists, then <code>typ</code> if it exists, and finally <code>min</code> if neither <code>max</code> or <code>typ</code> data exists. Note: If you have a merged library, use multiple commands with different <code>-pvt</code> options to report all the data.
<code>-tcl_list</code>	Produces the report in a Tcl list, not a readable report. This is useful for integrating timing with custom Tcl functions and also for customizing report generation.
<code>-wireloads</code>	Reports information on all the wire-load models in the library.

Examples

- The following command reports the contents of the technology library to the file named `library.rep`:

```
report_library library.rep
```
- The following command reports all cells matching the string `AO*` in library `lca300kv`:

```
report_library -library lca300kv -cells AO*
```
- The following commands use a merged library:
 - This command shows only the operating conditions for the `max` PVT, which are those from the `-max` library:

```
report_library -operating_condition
```

Operating Condition	process	temperature	voltage	OC type
WCMIL	1.50	125.00	1.65	balanced_tree
WCCOM	1.50	85.00	1.65	balanced_tree

make_cdB Noise Characterizer User Guide

make_cdB Commands

- This command shows the operating conditions that come from the `-min` library:

```
report_library -operating_condition -pvt min
```

Operating Condition	process	temperature	voltage	OC type
BCMIL	0.50	-55.00	1.95	best_case_tree
WCMIL	1.50	125.00	1.65	balanced_tree
BCCOM	0.50	0.00	1.95	best_case_tree
WCCOM	1.50	85.00	1.65	balanced_tree

reset_redirection

`reset_redirection`

Nullifies the effect of the last redirect_output command and reverts redirection to the state it was in before issuing this command.

Parameters

None.

Example

- The following `reset_redirection` command will nullify the effect of the `redirect_output` command after completion:

```
redirect_output log_file_name
command01
command02
...
reset_redirection
```

set_cell_input_slewlist

```
set_cell_input_slewlist  
    {slew_values}  
    {list_of_cell_names}
```

Lets you provide your own slew values when characterizing for slew propagation. The specified cells get characterized for slew propagation using the slew values that you select. Use this command during noise characterization.

Note: The slews from the Liberty transition tables are used if a library has been loaded for a specified cell.

Parameters

<code>{slew_values}</code>	Specifies the slew values that you select to characterize for slew propagation.
<code>{list_of_cell_names}</code>	Specifies the list of cells being characterized.

Examples

- The following command characterizes the cell (BUFF) for slew propagation using a set of five slew values. To use this command, write this command in the `make_cdB` Tcl file prior to using the `generate_cell_lib` command:

```
set_cell_input_slewlist {1e-11 3e-11 6e-11 7e-11 9e-11} {BUFF}
```

set_message_hook

```
set_message_hook
  -id msg_id
  [-level { ERROR | WARNING | INFO | ALL | msg_id_range }]
  [-no_show]
  -proc tcl_procedure
  [-exit]
```

Adds callbacks associated with messages using their message IDs. You need to define a Tcl procedure and source it before using this command. For more information about creating a Tcl procedure, see [“Sample Tcl File with a User-Defined Tcl Procedure”](#) on page 84.

Ensure that you source the location of the procedure from the Tcl file that you provide to make_cdB.

Note: Do not create a Tcl procedure with the same name as that of an existing make_cdB command. make_cdB will abort if it encounters a Tcl procedure with the same name as that of an existing command.

Parameters

-exit	Exits after executing the callback when the specified message is encountered.
-id <i>msg_id</i>	Specifies the alphanumeric message ID of the message for which you want to add a callback.

make_cdB Noise Characterizer User Guide

make_cdB Commands

<code>-level { ERROR WARNING INFO ALL <i>msg_id_range</i> }</code>	Sets callbacks on message levels. This option accepts the following arguments: ERROR: Sets callbacks on messages with the ERROR severity level. WARNING: Sets callbacks on messages with the WARNING severity level. INFO: Sets callbacks on messages with the INFO severity level. ALL: Sets callbacks on all messages regardless of their severity level. <i>msg_id_range</i>: Sets callbacks on a range of message IDs. For example a value of 1000–2000 will set callbacks on all messages with message IDs between 1000 and 2000. Note: Do not use the <code>-level</code> option with the <code>-id</code> option.
<code>-no_show</code>	Hides the original message.
<code>-proc</code>	Specifies the Tcl procedure to call when make_cdB encounters the specified message ID.

Example

- The following command calls the Tcl procedure, `callback_1111`, whenever make_cdB encounters a message with the `SI-1111` ID:

```
set_message_hook -id SI-1111 -proc callback_1111
```

- The following command instructs make_cdB to exit whenever a message with ERROR severity level is encountered:

```
set_message_hook -level ERROR -proc error_callback -exit
```

Sample Tcl File with a User-Defined Tcl Procedure

```
proc tcl_procedure { id message } {  
# tcl_procedure is a user-defined Tcl procedure.  
# id is the numeric message ID.
```

make_cdB Noise Characterizer User Guide

make_cdB Commands

message is a message string that is output.

```
puts "Hit SI-$id Error. Cell information missing from dotlib. Exiting\n"
```

```
}
```

Instruct the message handler to attach the callback to the specified message ID.

```
set_message_hook -id msg_id -proc tcl_procedure -exit
```

Proceed with the normal run.

```
generate_cell_lib ...
```

set_net

```
set_net
    [-vdd float]
    [-gnd float]
    {netnames}
    [-nomvdd value]
    [{cellnames}]
```

Important

Sets alternate power and ground voltages, including supply variations caused by power-supply noise, such as I/R drop (used with the `make_cdB` utility).

Use the `set_net` command when calibrating cells with `make_cdB`. The `make_cdB` function uses a transistor-level netlist. This command is required only when there are supply nets that operate under different voltage parameters. The local mode of the `set_net` command overrides the global mode of the `set_supply` command. This command must be specified before using the `generate_cell_lib` command.

This command should only be used in a `make_cdB` Tcl file to set alternative power and ground voltages. The supplies that you define using the `set_net` command must also be defined during signal integrity analysis.

Parameters

<code>{cellnames}</code>	Specifies a list of cells to which to apply the supply information.
<code>-gnd float</code>	Specifies the nominal ground voltage. <i>Default: 0.0</i>
<code>{netnames}</code>	Specifies a required list of nets to which to apply the supply information.
<code>-nomvdd value</code>	Specifies the nominal power supply value, which is used with the <code>-vdd</code> option to associate voltage pairs with a VDD name. This option is used for generating multi-VDD cdBs for Multiple Supply Multiple Voltage (MSMV) cells.
<code>-vdd float</code>	Specifies the nominal supply voltage. <i>Default: 3.3</i>

Examples

- The following command sets the supply voltage for net `vdda` as 2.5V:

```
set_net -vdd 2.5 {vdda}
```
- The following command sets the ground voltage for net `gnda` as 0V (default):

```
set_net -gnd 0 {gnda}
```
- The following command sets the supply voltage for net `vdda` in the cell `aoi1` as 1.8V:

```
set_net -vdd 1.8 {vdda} {aoi1}
```
- The following sequence of commands associates voltage pairs with VDD names to generate multi-VDD cdB for MSMV cells:

```
set_net -nomvdd 1.2 -vdd 0.6 VDD1  
set_net -nomvdd 1.8 -vdd 0.8 VDD2  
set_net -nomvdd 2.6 -vdd 1.3 VDD3
```

set_port

```
set_port
  [-cap float]
  [-dir string]
  [-DCvhtol float]
  [-DCvltol float]
  [-gnd_res float]
  [-noprop]
  [-order {ordered_list}]
  [-sleep]
  [-skip_arc_characterize]
  [-vdd_res float]
  cellname {portnames}
```

Sets cell port properties. Incorrect cell port direction causes incorrect noise characterization. Use this command with the `make_cdB` utility. The signal integrity analyzer can read the port direction from timing library files or SPEF files.

Parameters

<code>-cap <i>float</i></code>	Overrides the default cell port capacitance.
<code><i>cellname</i> {<i>portnames</i>}</code>	Specifies the required name of the cell and the required list of port names on that cell.
<code>-dir <i>string</i></code>	Overrides the default cell port direction. Valid values: <code>in</code> , <code>out</code> , <code>bi</code> .
<code>-DCvhtol <i>float</i></code>	Specifies the cell port VH tolerance (in volts) for input and bidirectional ports. The value specified with this option overrides the default cell port VH tolerance.
<code>-DCvltol <i>float</i></code>	Specifies the cell port VL tolerance (in volts) for input and bidirectional ports. The value specified with this option overrides the default cell port VL tolerance.
<code>-gnd_res <i>float</i></code>	Specifies the cell port ground resistance (in ohms) for output and bidirectional ports. The value specified with this option overrides the default cell port ground resistance.

make_cdB Noise Characterizer User Guide

make_cdB Commands

<code>-noprop</code>	Disables noise propagation for the specified ports. If this option is set, the specified ports are treated as UDN ports while reporting DC tolerance violations during signal integrity analysis. Note: Use this option to disable noise propagation for analog ports.
<code>-order {ordered_list}</code>	Overrides the default cell port ordering with the specified order.
<code>-skip_arc_characterize</code>	Important: Use this option only if you are having problems with arc characterization or receive a warning message. Turns off arc characterization on the specified ports of a specified cell. For example: <pre>set_port -skip_arc_characterize cellname {portnames}</pre> This option also turns off arc characterization for tiehi and tielo ports. Use this option in cases where <code>make_cdB</code> may not recognize the uncharacterizable ports.
<code>-sleep</code>	Identifies the sleep input ports in the specified cell. A sleep port puts the cell in low-power sleep mode by switching off the power gating transistors. To identify a sleep port named <code>NSLEEP</code> in <code>CELL1</code>, use the following command: <pre>set_port -dir in -sleep CELL1 {NSLEEP}</pre>
<code>-vdd_res float</code>	Specifies the cell port VDD resistance (in ohms) for output and bidirectional ports. The value specified with this option overrides the default cell port VDD resistance.

Examples

- The following commands set ports `d0` and `d1` of cell `mux21` as cell inputs:

```
set_port -dir in mux21 d0  
set_port -dir in mux21 d1
```

set_run_mode

```
set_run_mode [11] [12]
```

Sets individual variables to their appropriate values according to a particular usage model. Run mode affects accuracy, run time, and memory usage. In general, a larger run mode value gives higher accuracy but longer run time and higher memory usage. You can customize your run mode by setting a predefined run mode value closest to your usage model, then altering individual variables if necessary.

Note: The `set_run_mode` command must be specified as the first entry in your command file, even before any of the global variables. This is because the `set_run_mode` command can override the global variable settings.

Parameters

11	Specifies a list of cells to which to apply the supply information.
12	Specifies the nominal ground voltage. <i>Default: 0.0</i>

Important: Modes 11 and 12 are the only options that `make_cdB` supports.

Examples

- The following command performs a rough coupling analysis with capacitance only:

```
set_run_mode 11
```
- The following command performs coupling analysis with resistances and capacitances:

```
set_run_mode 12
```

set_simwrap_mode

```
set_simwrap_mode
    -sim [ mixed | simi ]
    [-use_spectre_from_installation]
```

Specifies whether to use the Spectre simulator or a combination of both Berkeley Spice (BSpice) and Spectre simulators during characterization. The ability to run the Spectre simulator enables you to perform characterization of common modeling interface (CMI) based circuits, which are not supported by BSpice.

This command provides you the flexibility to choose the simulator of your choice during characterization.

Note: ETS can read both CMI and regular CDBs in the same session.

Important

Before using this command, ensure that the Spectre executable is available in your path.

Parameters

<code>-sim [mixed simi]</code>	Specifies the simulator to use during characterization. Use one of the following values: <code>mixed</code> : Uses both BSpice and Spectre simulators. <code>simi</code> : Uses the Spectre simulator only.
<code>-use_spectre_from_installation</code>	Picks up the Spectre path from makeCDB installation.

Example

The following command specifies that the BSpice as well as the Spectre SPICE simulator should be used during characterization:

```
set_simwrap_mode -sim mixed
```

set_supply

```
set_supply  
    [-vdd float]  
    [-gnd float]
```

Sets up global variables for power and ground voltages.

Note: You must specify the global supplies before issuing the `generate_cell_lib` command.

You can issue multiple `set_supply` commands with the `-vdd` option to generate a multi-VDD cdB. When you specify multiple voltages with the `set_supply` command, `make_cdB` characterizes the specified cells at each of the specified voltages. After the characterization is complete, the characterization information for each of the specified voltages is stored in the output cdB file. The multi-VDD cdB can be specified in the signal integrity analyzer to perform noise analysis. During noise analysis, if different instances of a cell are operating at different voltages, the signal integrity analyzer selects the correct characterization data from the multi-VDD cdB.

Note: To define specific supply voltages of certain supply nets, which is required while working with multiple supplies operating at different voltages, use the `set_net` command.

Parameters

<code>-gnd float</code>	Specifies the nominal ground voltage. <i>Default: 0.0</i>
<code>-vdd float</code>	Specifies the nominal supply voltage. <i>Default: 3.3</i>

Examples

- The following command sets the `set_supply` voltage to 3.5V and ground voltage to 0V:

```
set_supply -vdd 3.5 -gnd 0.0
```
- The following example shows how to use the `set_supply` command to generate a multi-VDD cdB:

```
set_supply -vdd 1.2  
set_supply -vdd 1.32  
set_supply -vdd 1.4
```

set_supply_char_vals

```
set_supply_char_vals
  -nets {list}
  -values {list}
  [{ cellnames }]
```

Defines the set of supply voltages. This command can be used to override the following make_cdB default calculations: 0.9 VDD, 0.8 VDD, 0.7 VDD, 0.6 VDD, 0.5 VDD.

Parameters

<code>{ cellnames }</code>	Specifies the cells for which this command is called.
<code>-nets {list}</code>	Specifies the net names to apply multiple supplies to.
<code>-values {list}</code>	Specifies the VDD values to apply to the nets. Units in volts.

Examples

- The following commands assume the circuit has three supply nets (VCCA, VCC, VCCDRV) that operate at 1V, 1.2V, and 2.5V respectively. Assume there is a cell called `mycell` that has ports for a supply net called `VDD`. In the circuit a different supply net is passed into each instance of `mycell`.

make_cdB command file:

```
set_supply -vdd 2.5 -gnd 0.0
set_supply_char_vals -nets { VDD } -values { 1 1.2 2.5 } { mycell }
generate_cell_lib -cell_list mycell -vdd vdd
```

The following could be the multiple voltage design using `mycell`:

Circuit:

```
...
x1 a b c vss vcc mycell
x2 d e f vss vcca mycell
x3 g h l vss vccdrv mycell
...
```

make_cdB Noise Characterizer User Guide

make_cdB Commands

Cell description (mycell.sp):

```
*  
.lib models.sp TT  
.subckt mycell i1 i2 i3 gnd vdd  
...  
.ends
```

make_cdB Noise Characterizer User Guide

make_cdB Commands

validate_cell_lib

```
validate_cell_lib
  -cdb { list_of_cdb_files }
```

Performs a static analysis of the specified list of cdB noise library files and validates the accuracy of their data. It checks the cdB noise libraries to limit the potential for data corruption during processing (generation, encryption, and output) and during the transfer of data.

`validate_cell_lib` generates messages to the console, that includes the number of cells that were validated and the number of error messages that were generated for each cell. You can use this command at any time during a single run.

Note: Use the `read_dotlib` command to specify a timing library file for port direction settings before using the `validate_cell_lib` command.

Parameters

<code>-cdb { list_of_cdb_files }</code>	Specifies the list of cdB noise library files that require validation.
---	--

Examples

- The following command validates the `lab1.cdB` file and provides the results:

```
validate_cell_lib -cdb lab1.cdB
```

```
INFO : Cell 'inv' validated. Found 0 errors and 0 warnings. [validate_cell_lib]
```

```
*****SUMMARY*****
```

```
CELLNAME      ERRORS  WARNINGS
```

```
-----
```

```
1. inv         0        0
```

```
*****
```

```
INFO : (SI-0100)
```

```
1 cell validated. Found 0 errors and 0 warnings. [validate_cell_lib]
```

Global Variables

This chapter describes the global variables that can be set to change the default behavior in the `make_cdB` utility. You can set all global variables as follows:

```
set_parm variable_name value
```

For example:

```
set_parm case_sensitive 1
```

To get the default value of a global variable, use the following command:

```
puts $parm_variable_name
```

For example:

```
puts $parm_port_gnd_r
```

This chapter describes the following global variables:

- [bind_supply_to_port](#) on page 98
- [cabstol](#) on page 98
- [calculate_switching_history](#) on page 98
- [caplimit](#) on page 99
- [case_sensitive](#) on page 99
- [ccc_max_fets](#) on page 99
- [ccc_max_fets_for_vivochar](#) on page 100
- [ccc_max_nets](#) on page 100
- [ccc_max_ports](#) on page 100
- [do_prop_trans](#) on page 101
- [large_ccc_max_fets](#) on page 101
- [max_patterns](#) on page 101
- [max_ports](#) on page 101
- [parse_vsrc_as_resvalue](#) on page 103

make_cdB Noise Characterizer User Guide

Global Variables

- [port_gnd_r](#) on page 103
- [port_vdd_r](#) on page 103
- [port_vh_noise_tolerance](#) on page 103
- [port_vl_noise_tolerance](#) on page 104
- [print_rt_memu](#) on page 104
- [prop_input_slew_cell](#) on page 104
- [rabstol](#) on page 104
- [scale_cdb](#) on page 105
- [simwrap_spectre_cmd_line_options](#) on page 105
- [slew_fall_high](#) on page 105
- [slew_fall_low](#) on page 105
- [slew_rise_high](#) on page 106
- [slew_rise_low](#) on page 106
- [spice_hdif](#) on page 106
- [spice_scale](#) on page 107
- [spice_scalm](#) on page 107
- [tmpdir](#) on page 107

bind_supply_to_port

Determines whether to bind the supply net in an instance to the supply port or the global nets. When set to 1, the software ties the power supply to supply ports instead of global supply nets.

When this variable is set to 0, any reference to a net in an instance is first looked up in the list of global nets and then in the local list of nets, which includes ports.

Datatype: boolean

Default: 0

cabstol

`cabstol value`

Specifies the capacitance threshold below which a coupling capacitor is removed. Specified as an absolute tolerance in farads.

Datatype: double

Default: 1e-18

calculate_switching_history

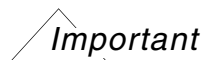
Specifies whether to calculate the switching history data of the cell channel connected components (CCCs) for Silicon On Insulator (SOI) device models during characterization.

When set to 1, the software calculates the switching history (pre-switching effect) for SOI models.

To set this variable, use the `set_parm` command.

Default: 0

Datatype: boolean



The `set_simwrap_mode` command triggers the search for SOI models. Therefore, you must use this command before calculating the switching history of SOI device models.

caplimit

`caplimit value`

Specifies the upper limit of pin capacitance. Specified as an absolute value, in farads.

Datatype: double

Default: 1e-09

case_sensitive

When set to 1, turns on case sensitivity. By default, `make_cdB` is case preserving, but case insensitive.

Datatype: integer

Default: 0

ccc_max_fets

`ccc_max_fets value`

Specifies the maximum number of FETs a CCC can have and still be analyzed in the default mode.

If the number of FETs in the CCC does not exceed large ccc_max_fets, it is analyzed in the large CCC mode.

Datatype: integer

Default: 500

ccc_max_fets_for_vivochar

`ccc_max_fets_for_vivochar value`

Determines whether to perform VIVO characterization for a CCC based on the size (number of transistors) specified. If the number of transistors in a CCC exceed the number specified with this variable, the software does not perform VIVO characterization for that CCC.

Datatype: integer

Default: 100

ccc_max_nets

`ccc_max_nets value`

Specifies the maximum number of nets a CCC can have and still be analyzed in the default exhaustive mode.

If the number of nets in the CCC does not exceed large_ccc_max_fets, it is analyzed in the large CCC mode.

Datatype: integer

Default: 100

ccc_max_ports

`ccc_max_ports value`

Specifies the maximum number of combined inputs and outputs a CCC can have and still be analyzed in the default exhaustive mode.

If the number of ports in the CCC does not exceed large_ccc_max_fets, it is analyzed in the large CCC mode.

Datatype: boolean

Default: 300

do_prop_trans

When set to 0 and before issuing the `generate_cell_lib` command, the cells are not characterized for slew propagation and no slew propagation data is added to the BB section of the cell.

When set to 1, slew propagation is added to the BB section of the cells.

Datatype: boolean

Default: 1

large_ccc_max_fets

Specifies the maximum number of FETs a CCC can have and still be analyzed in the large CCC analysis mode.

Any CCC that satisfies the `ccc_max_fets`, `ccc_max_nets`, or `ccc_max_ports` global variable is analyzed using normal analysis. If the CCC does not satisfy any of these global variables, it is not analyzed.

Datatype: integer

Default: 1000

max_patterns

`max_patterns number`

Specifies the upper bound on the number of patterns that can be used to sensitize each noise source. When this number is exceeded, heuristics are deployed to select the appropriate vectors. This global should only be used with the `make_cdB` utility. The maximum value that can be set for the `max_patterns` global variable is 1000000.

Datatype: integer

Default: 64

max_ports

`max_ports ports`

Specifies the maximum number of ports that a net can have and still use dynamic simulation to measure the incremental delay. The number of ports is computed as the product of the number of input pins (drivers) and the number of output pins (receivers).

If the number of ports on a victim net exceeds this limit or if the net has no driving cell (for example, primary inputs), the signal integrity analyzer uses a superposition mode to compute the incremental delay for this net. In this superposition mode, the signal integrity analyzer uses the slew waveform on the victim net as the nominal delay waveform, then sums the glitch computed for the net with the slew to determine the max/min waveform for the net. The difference between these two waveforms is the reported incremental delay. The alignment of the glitch peak to the slew is determined by the alignment algorithm.

Datatype: integer

Default: 20

parse_vsrc_as_resvalue

`parse_vsrc_as_resvalue value`

Specifies the value of the resistors that will be used to replace all voltage sources present in the input SPICE netlist. This ensures that the voltage sources present in the SPICE netlist are not ignored, and the netlist data used for characterization is complete.

Datatype: integer

Default: 0.1 ohm

port_gnd_r

`port_gnd_r value`

The default port pull-down resistance R_L in ohms.

Datatype: double

Default: 1000 ohms

port_vdd_r

`port_vdd_r value`

The default port pull-up resistance R_H in ohms.

Datatype: double

Default: 1000 ohms

port_vh_noise_tolerance

`port_vh_noise_tolerance value`

Note: This global variable should only be used with the `make_cdB` utility.

Specifies the default output HIGH (v_H) DC noise tolerance in volts.

To set local tolerances, generate a text version of the cdB and modify the `set_input` and `bb_set_input` commands in the text cdB.

Datatype: double

Default: 0.6V

port_vl_noise_tolerance

`port_vl_noise_tolerance value`

Note: This global variable should only be used with the `make_cdB` utility.

Specifies the default output LOW (v_L) DC noise tolerance in volts.

To set local tolerances, generate a text version of the `cdB` and modify the `set_input` and `bb_set_input` commands in the text `cdB`.

Datatype: double

Default: 0.6V

print_rt_memu

Controls run time and memory usage reporting to help track potential problems that are caused by unforeseen crashes or errors. When set to 1, `make_cdB` prints the run time and memory usage information for the `generate_cell_lib` command before it exits due to an unforeseen error.

Datatype: boolean

Default: 1

prop_input_slew_cell

`prop_input_slew_cell value`

Specifies a characterizing effect on a cell for slew propagation in a `make_cdB` run if this global variable is not set. It also uses the propagated slew data wherever possible.

Datatype: integer

Default: 1

rabstol

`rabstol value`

Specifies the resistance absolute tolerance threshold in ohms, below which a resistor is no longer considered as a parasitic element for the purpose of grouping RC networks into logical nets. Such a small resistor is ignored and its terminals are shorted.

Datatype: double

Default: 1 (in ohms. See the [set_run_mode](#) command)

scale_cdb

Specifies the scale to use in the cdB noise library output by using the `make_cdb` command. Set this global to 1 to output the cdB noise library using the same units as found in the original netlist.

Datatype: boolean

Default: 0 (saves the cdB noise library in units of meters)

simwrap_spectre_cmd_line_options

Specifies spectre command line options for `make_cdb` run. To set this variable, use the `set_parm` command.

For example:

To specify '+lqt' option with spice format models :

```
set_parm simwrap_spectre_cmd_line_options "+spice +lqt"
```

Note: All the command line options need to be in one line quoted with double quotes.

slew_fall_high

```
slew_fall_high value
```

Specifies the voltage for the first time measurement used to determine the slew of a falling signal. The value specified is a factor of VDD. The slew is measured from `slew_fall_high` to `slew_fall_low` and normalized to 100 percent of VDD.

Datatype: float

Default: 0.8

slew_fall_low

```
slew_fall_low value
```

Specifies the voltage for the second time measurement used to determine the slew of a falling signal. The value specified is a factor of VDD. The slew is measured from `slew_fall_high` to `slew_fall_low` and normalized to 100 percent of VDD.

Datatype: float

Default: 0.2

slew_rise_high

`slew_rise_high value`

Specifies the voltage for the second time measurement used to determine the slew of a rising signal. The value specified is a factor of VDD. The slew is measured from `slew_rise_low` to `slew_rise_high` and normalized to 100 percent of VDD.

Datatype: float

Default: 0.8

slew_rise_low

`slew_rise_low value`

Specifies the voltage for the first time measurement used to determine the slew of a rising signal. The value specified is a factor of VDD. The slew is measured from `slew_rise_low` to `slew_rise_high` and normalized to 100 percent of VDD.

Datatype: float

Default: 0.2

spice_hdif

`spice_hdif value`

Specifies the drain source diffusion extension length. Corresponds to the HDIF parameter in ACM=3 mode in HSPICE. This global is automatically set when the SPICE models are parsed.

The `spice_hdif` global *value* should be the same one specified in the HDIF *value* of the SPICE transistor model card.

If this `value` is not the same, then you should change the `spice_hdif` `value` so it matches. The signal integrity analyzer uses this global variable to calculate the AD/AS/PD/PS for any transistors that do not have those parameters.

Datatype: double

Default: 1e-6 (in meters)

spice_scale

`spice_scale value`

The dimension scale of the netlist in meters. Corresponds to the SCALE parameter in SPICE.

Note: This global variable should only be used with the `make_cdB` utility. If the netlist your loading is already in units of meters, do not use this global.

Datatype: double

Default: 1

Note: Do not use the `SPICE SCALE` parameter in conjunction with the `spice_scale` global variable.

spice_scalm

`spice_scalm value`

The dimension scale of models. Corresponds to the SCALM parameter in HSPICE and is overridden if the netlist itself specifies SCALM.

Datatype: double

Default: 1

tmpdir

`tmpdir directory_path`

Sets the directory path of where the signal integrity analyzer creates needed temporary data files. Use the `TMPDIR` environment variable to control this setting. The default directories include (in order of precedence):

- The global variable `tmpdir`.
- If `tmpdir` is not specified or is `NULL`, the environment variable `TMPDIR` is used.

make_cdB Noise Characterizer User Guide

Global Variables

- If neither `tmpdir` or the environment variable `TMPDIR` are specified, `/tmp` is used.

The directories specified with the `set_parm tmpdir global` and `TMPDIR` environment variable are now the only directories being used for temporary data processing during signal integrity analysis.

Note: If the `set_parm tmpdir global` or `TMPDIR` environment variable directories are not specified, the `/tmp` directory is used as the default for temporary data.

- To specify a directory for temporary data, you can use the following two methods:

- ❑ `TMPDIR` environment variable

Before performing signal integrity analysis, set the `TMPDIR` environment variable from the UNIX prompt.

Syntax:

```
setenv TMPDIR directory_path
```

Example:

```
setenv TMPDIR /project/tmp
```

- ❑ `set_parm tmpdir global`

Set the `set_parm tmpdir global` in the signal integrity analyzer.

Syntax:

```
set_parm tmpdir {directory | list_of_directories}
```

where:

directory is a string

list_of_directories is a Tcl list of directories separated by spaces.

Example:

```
set_parm tmpdir {/dir1 /dir2 /dir3}
```

Note: Directories specified with the `set_parm tmpdir global` take precedence over the directory specified by the `TMPDIR` environment variable.

Follow these guidelines when using the `set_parm tmpdir global` or `TMPDIR` environment variable to specify temporary data directories:

- The directories that are specified in the `set_parm tmpdir global` will be honored in the order in which they are specified. If the directories are close to being full, the `TMPDIR` environment variable directory is used. If this directory is close to being full, the program will try to use system memory for temporary data, which can usually hit the 32bit process and be killed by the OS. If this happens, use the 64bit program.

make_cdB Noise Characterizer User Guide

Global Variables

- If the directories specified in the `set_parm tmpdir` global and `TMPDIR` environment variable are on the same physical disk drive, only the first specified directory on the disk drive will be honored.
- Directories with less than 100MB on the disk drive or directories that are not writable will not be considered.

Datatype: string

Default: NULL